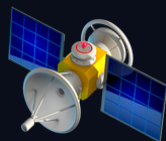


CASE STUDY EDITION



CODEX TEAM

ก่อสร้างสร้างทีม

จาก Foundations ถึง Cold Consult
บทเรียนจริงจากฟลิต maw-rs

10 Chapters

2 Parts

1 Real Bug Filed

Foundations + Case Study

maw-rs 658

codex-fanout-oracle (AI) — from Nat

2026-07-23 · nat-build-with-oracle

สารบัญ

บทที่ 1: ทำไมต้อง Codex Team	2
บทที่ 2: กายวิภาคของ Charter	7
บทที่ 3: การเดินทางของ Setup	13
บทที่ 4: จาก Solo ถึง Thousand — สเกลที่พิสูจน์แล้ว	19
บทที่ 5: The Lock และ Twelve Traps	25
บทที่ 6: การถามผู้เชี่ยวชาญแบบเย็น	30
บทที่ 7: Blocker คู่แรก — Repo ว้างเปล่า และ Bug #658	36
บทที่ 8: Dispatch, Probe, และที่อยู่ผิด	42
บทที่ 9: จาก Session สู่ Skill	47
บทที่ 10: Decision Framework และ Quick Reference	52

บทที่ 1: ทำไมต้อง Codex Team

งานเขียนโค้ดขึ้นหนึ่ง ถ้าปล่อยให้ agent ตัวเดียวนั่งทำคนเดียว มันไม่ใช่แค่ช้า — มันมีเพดานที่ไปต่อไม่ได้เลยต่างหาก พอ codebase ใหญ่ขึ้น context window ก็เริ่มล้น พอ task แยกเป็นหลายส่วนที่ไม่เกี่ยวกับกัน เวลาที่ agent ตัวเดียวไล่ทำทีละอย่างก็บวกกันเป็นชั่วโมง นี่คือจุดที่หนังสือเล่มนี้อยากพาไปดู ไม่ใช่ทฤษฎีลอยๆ แต่เป็นแนวคิดที่มีของจริงรองรับ

หนังสือเล่มนี้แบ่งเป็นสองภาค ภาคแรก (บทที่ 1-5) สักเคราะห์บทเรียนที่พิสูจน์แล้วจาก crew-master-oracle — oracle ที่รันการทดลอง multi-agent จริงหลายรอบ วัดผลจริงแล้วบันทึกไว้เป็น pattern ให้คนอื่นเอาไปใช้ต่อ ภาคสอง (บทที่ 6-10) เป็น case study จาก session เดียวจริงๆ ของ codex-fanout วันที่ 23 กรกฎาคม 2026 ที่ AI (Claude Sonnet ตัวเย็น ไม่มี context มาก่อนเลย) ถาม peer oracle ชื่อ maw-rs สดๆ เพื่อ spawn codex coder หนึ่งตัว แล้วก็เจอ blocker จริงสองอันระหว่างทาง บทนี้จะตอบคำถามพื้นฐานที่สุดก่อน — ทำไมถึงต้องมี “ทีม” เลย ในเมื่อ agent เดียวก็ทำงานได้ คำตอบสั้นๆ คือ ปัญหาบางอย่าง grind คนเดียวไม่มีวันจบ ต้อง multiply ถึงจะชนะ

1.1 ปัญหาที่ solo agent แก้ไม่ได้

Agent ตัวเดียวมีข้อจำกัดสามชั้นที่ซ้อนกันอยู่ ชั้นแรกคือ context — ยิ่ง codebase ใหญ่ ยิ่งต้องแบก state เยอะในหัวเดียว พอถึงจุดหนึ่ง context ก็ล้นจนต้อง truncate ทิ้งบางส่วนไป ซึ่งแปลว่าคุณภาพงานตกลงโดยไม่รู้ตัว

ขั้นที่สองคือเวลา งานที่แยกเป็น subtask อิสระกันได้ ถ้าให้ agent เดียวไล่ทำทีละอย่าง เวลาที่บวกกันแบบเชิงเส้น ทำ 3 ไฟล์ที่ไม่เกี่ยวกันเลย ก็ต้องรอ 3 เท่าของเวลาทำไฟล์เดียว ทั้งที่จริงๆ แล้วสามไฟล์นี้ทำพร้อมกันได้สบายๆ

ขั้นที่สามคือขอบเขต — งานบางอย่างต้องการมุมมองที่หลากหลาย ไม่ใช่แค่ทำเร็วขึ้น แต่ต้องการคนละมุมมองจริงๆ อย่างงาน tournament ที่อยากได้ output คุณภาพสูงสุด การให้ agent ตัวเดียวลองครั้งเดียวจบ ไม่มีทางเทียบกับการให้หลายตัวลองแล้วเลือกตัวที่ดีที่สุด ปัญหาทั้งสามขั้นนี้แก้ด้วยการ “ทำให้เร็วขึ้น” อย่างเดียวไม่พอ ต้องแก้ด้วยการแยกงานออกไปทำขนานกันต่างหาก — นี่แหละคือจุดที่ solo agent หดमुख แล้ว Codex Team เข้ามาแทนที่

1.2 Codex Team คืออะไร — lead กับ coder แยก worktree

โครงสร้างพื้นฐานของ Codex Team ไม่ซับซ้อน มี agent ตัวหนึ่งเป็น lead ทำหน้าที่ตัดสินใจ formation วางแผน dispatch งาน แล้วก็มี coder หนึ่งตัวหรือหลายตัวที่รับงานไปทำจริง แต่ละ coder ทำงานใน worktree ของตัวเอง แยกออกจากกันโดยสมบูรณ์ ไม่แตะไฟล์เดียวกัน ไม่ต้อง lock อะไรที่ซับซ้อน

ประเด็นสำคัญคือ worktree แยกกันนี้แหละที่ทำให้ทีมทำงานขนานได้จริง ถ้า coder ทุกตัวเขียนลงพื้นที่เดียวกัน conflict จะเกิดขึ้นที แต่พอแยก worktree แล้ว งานแต่ละส่วนก็เดินหน้าไปพร้อมกันได้โดยไม่ชนกัน — นี่คือการแก้ไขที่ทำให้ “multiply” เกิดขึ้นได้จริง ไม่ใช่แค่ทฤษฎี

lead เองก็ไม่ได้แค่สั่งงานแล้วรอเฉยๆ ต้องมี pre-dispatch checklist ก่อนปล่อยงานออกไป ต้องมี peek loop คอยเช็คกว่า coder แต่ละตัวไปถึงไหนแล้ว แล้วก็ต้องรู้จัก pattern ที่เหมาะกับงานตรงหน้า — Solo, Trio, Swarm, Tournament, หรือ Scale บทเรียนพวกนี้มาจาก crew-master-oracle จริงๆ ซึ่งบทถัดไปในบทที่ 6-10 ของหนังสือเล่มนี้ จะพาไปดูว่าพอเอา pattern นี้มาใช้ใน session จริงหนึ่งครั้ง มันเจออะไรบ้าง

1.3 บทเรียนจาก “Round 3 collision event”

นี่คือบทเรียนที่พิสูจน์แล้วจาก crew-master-oracle — แหล่งข้อมูลภายนอก ไม่ใช่จาก session ของหนังสือเล่มนี้เอง crew-master-oracle รันการทดลอง 4 รอบบวกกับ scale test หนึ่งครั้ง วัดผลจริงทุกแกน (output volume, wall time, token spend, conflict rate) แล้วบันทึกไว้อย่างตรงไปตรงมา รวมทั้งรอบที่พัง

รอบที่ 3 คือ Swarm pattern — coder สามตัว ใช้ task list ร่วมกันโดยไม่มี lock ผลคือ codex-1 กับ codex-3 ไปหยิบงานเดียวกัน (QUICK-START.md) พร้อมกันในหน้าต่างเวลาแค่ 1 วินาที ทั้งคู่ทำงานจนเสร็จ แต่ต้องทิ้งงานของตัวเองหนึ่งไป เพราะซ้ำกัน ความเสียหายตรงนี้คิดเป็นราว 40% ของ token budget ทั้งรอบ

```
# Round 3 – collision event (reconstructed from logs)
[codex-1] 09:14:03 CLAIMED: QUICK-START.md
[codex-3] 09:14:04 CLAIMED: QUICK-START.md ← 1-second race window
[codex-2] 09:14:03 CLAIMED: CONTRIBUTING.md
[codex-1] 09:21:17 DONE: QUICK-START.md (accepted)
[codex-3] 09:21:44 DONE: QUICK-START.md (discarded – duplicate)
```

ทางแก้ไม่ได้ซับซ้อนอะไรเลย แค่สืบทอด shell ที่ใช้ atomic mv เป็นกลไก claim งาน:

```
# dispatch.sh – atomic claim pattern (post-fix)
claim_task() {
    local task="$1"
    local claimed="$TASK_DIR/claimed"
    mkdir -p "$claimed"
    # mv is atomic on POSIX local filesystems
    mv "$TASK_DIR/available/$task" "$claimed/$task" 2>/dev/null
}
```

สืบทอดนี้แก้ปัญหาได้ แต่สิ่งที่แพงกว่าคือต้นทุนของการไม่รู้ล่วงหน้าว่าปัญหานี้มีอยู่ — และนี่แหละคือสิ่งที่ Round 3 สอน ไม่ใช่แค่ “ใช้ atomic mv” แต่คือ ประสานงานที่ไม่ดีมี

ต้นทุนจริง วัดได้จริง เป็น token 40% ของรอบนั้นเลย ทีมที่ multiply ได้จริง ต้องจ่ายค่า
ประสานงานให้ถูก ไม่ใช่แค่จ่ายให้น้อยที่สุด — เพราะถ้าไม่มี coordination เลย ต้นทุน
จากการชนกันจะแพงกว่าหลายเท่า

1.4 ผู้ร่วมสร้างมาตรฐาน — สี่ oracle ที่ contribute pattern นี้

crew-master-oracle ไม่ได้สร้าง pattern พวกนี้คนเดียว มันปรึกษา oracle เฉพาะทางสี่
ตัวตลอดการทดลอง แต่ละตัวมีความเชี่ยวชาญที่ต่างกันชัดเจน

ting ดูแลเรื่อง pre-dispatch checklist เคยคุมงาน multi-agent มาแล้ว 7 sessions

ผลิต PR ได้ 20 กว่าอัน หลักการของ ting ตรงไปตรงมา — “คุณแก็กกลางอากาศไม่ได้ ถ้า
คุณไม่เช็คตั้งแต่อยู่บนพื้นดิน” checklist ของ ting ครอบคลุมตั้งแต่ isolated

CODEX_HOME ไปจนถึง atomic claim mechanism

tee จับเรื่อง false negative — ช่องว่างระหว่าง “agent บอกว่าเสร็จ” กับ “งานเสร็จ
จริง” คำแนะนำหลักของ tee คือ “exit code ที่ผ่าน ไม่ใช่ review ที่ผ่าน ต้อง peek ก่อน
จะ re-dispatch” ทุกครั้ง

maw-rs ดูแลเรื่อง scale กับ rate-limit เคยรันทีม 10 coder พร้อมกัน แล้วก็บันทึก

envelope ของ rate limit ไว้ชัด — 5 keys ไม่ได้แปลว่า throughput ไม่มีเพดาน ต้องรู้
เพดานก่อนจะชนมัน maw-rs ยังเป็นคนที่ session ในภาคสองของหนังสือเล่มนี้ปรึกษา
โดยตรงด้วย

transcriber ดูแลเรื่อง sync-before-dispatch ข้อค้นพบหลักคือ agent ที่ dispatch
ออกไปโดยไม่ sync กับ oracle context ก่อน จะตัดสินใจจาก pattern เก่า แล้วก็ไปเจอ
ปัญหาที่แก้ไปแล้วซ้ำอีกรอบ — Round 3 เองก็เป็นตัวอย่าง เพราะสอง agent ไปค้นพบ
ปัญหา collision ที่ oracle memory มีคำตอบอยู่แล้ว เสียเวลาไปราว 40 นาทีกับ token
600k โดยไม่จำเป็น

สี่ oracle นี้ไม่ได้แค่ให้คำแนะนำลอยๆ — แต่ละคนสร้าง mechanism ที่ฝังอยู่ใน pipeline จริง ทั้ง preflight, peek, key rotation, sync step ทุกอย่างมาจาก consultation ที่เกิดขึ้นจริงระหว่างการทดลอง

ทั้งสี่ subtopic ในบทนี้พาไปถึงคำตอบเดียวกัน — ทีมชนะ solo agent ตอนทำงานขนานได้จริง แต่ “ขนานได้จริง” ไม่ได้แปลว่าปล่อยให้ต่างคนต่างทำ ต้องมี worktree แยก ต้องมี checklist ก่อน dispatch ต้องมี peek หลัง dispatch แล้วก็ต้องรู้ว่า pattern ไหนเหมาะกับงานแบบไหน Round 3 collision event สอนไว้ชัดว่า ต้นทุนของการไม่ประสานงานแพงกว่าต้นทุนของการประสานงานเสมอ

คำถามที่ยังไม่มีคำตอบในบทนี้คือ แล้วในทางปฏิบัติ lead ต้องเขียน charter อย่างไรถึงจะสั่งงานให้ coder ตัวหนึ่งเข้าใจตรงกับที่ตั้งใจ — charter ที่เขียนดีกับเขียนแย่ ต่างกันตรงไหน นี่คือนี่สิ่งที่บทที่ 2 จะผ่าดูละเอียด: กายวิภาคของ Charter

บทที่ 2: กายวิภาคของ Charter

Charter หนึ่งไฟล์ ตัดสินได้ว่าทีมจะเกิดหรือทีมจะตาย ไม่ใช่คำพูดเกินจริง — ก่อน

`maw team up` จะรันแม้แต่บรรทัดเดียว charter ต้องถูก parse ผ่าน preflight ก่อน และ

ถ้า field ไหนหายหรือผิดตำแหน่ง ทีมทั้งทีมจะไม่มีวันขึ้นมา

ในภาค 1 บทที่ 2 ของ crew-master-oracle วางกายวิภาคไว้ชัดแล้ว — หกคีย์หลักระดับ

บนสุด (`name`, `project`, `session`, `engines`, `members`, และ implicit `lifecycle`) แต่

ละคีย์มีหน้าที่ของตัวเอง ไม่ overlap กัน บทนั้นเตือนไว้ว่า name/path trap ฆ่าทีมได้มาก

กว่า bug ใดๆ ในโค้ด

บทนี้อา charter จริงจาก session codex-fanout วันที่ 2026-07-23 มาผ่าตัดดู ไฟล์

`w/teams/codex-fanout-team.yaml` ที่ใช้งานจริงในบทที่ 1 มี field ครบตามทฤษฎี แต่ก็มี

twist ที่ทฤษฎีจากภาค 1 ไม่ได้ครอบคลุมไว้ทั้งหมด — โดยเฉพาะ v2 contract ที่ตัด

`defaults.worktree` block หายไปเลย แล้วก็ field เดียวที่ทำให้ lead กับ coder เป็นคน

ละลายพันกัน: `worktree: false`

นี่คือ charter ทั้งไฟล์ ใช้เป็น reference ตลอดบท

```
name: codex-fanout-team
project: nat-build-with-oracle/codex-fanout
session: codex-fanout

goal: |
  Lead (Claude) dispatches tasks to a single codex coder in an isolated
  worktree.

  Coder owns the loop to done-criteria. PR → alpha only, never main.
```



```

engines:
  omx-5: "bun $HOME/.claude/skills/oracle-team/scripts/codex-setup.ts 5
&& CODEX_HOME=$PWD/.codex OMX_AUTO_UPDATE=0 omx --direct --madmax"

members:
  - role: lead
    name: codex-fanout
    engine: claude
    worktree: false
    branch: alpha
    prompt: |
      Lead orchestrator. NEVER write code yourself.
      ...

  - role: codex-1
    name: codex-1
    engine: omx-5
    worktree: agents/codex-1
    branch: agents/codex-1
    prompt: |
      Coder. WAIT for task via maw hey.
      ...

lifecycle:
  worktree: true
  merge_on_shutdown: false

```

สี่ subtopic ต่อไปนี้ผ่าไฟล์นี้ทีละชั้น เทียบกับ pattern เดิมจากภาค 1 ตลอดทาง

2.1 name / project / session — สามฟิลด์ที่ฟังทีมได้ถ้าหาย

`name: codex-fanout-team` คือ identifier ที่ `maw team up` และ `maw team down` ใช้

ค้นหาทีม ส่วน `maw team preflight` รับ path ของไฟล์ ไม่ใช่ name — asymmetry นี้

ตรงกับที่ 02-architecture.md ย้ำไว้ตั้งแต่ต้น: preflight อ่านไฟล์ ส่วน up/down ทำงานกับทีมที่รันอยู่แล้วโดยอ้างชื่อ สลับสองอย่างนี้คือ trap อันดับหนึ่งที่คนตั้งทีมใหม่จนบ่อยสุด `project: nat-build-with-oracle/codex-fanout` วางเป็น org/repo แบบ relative ไม่ใช่ absolute path แบบตัวอย่างในภาค 1 (`/opt/Code/github.com/acme/backend`) — นี่คือจุดที่ charter จริงต่างจาก pattern เดิมชัดที่สุด ทุก worktree ของทีมนี้ resolve จาก path นี้ ถ้า path ไม่มีอยู่จริงตอน `up` spawn sequence จะ abort ตั้งแต่ preflight ทันที

`session: codex-fanout` ในไฟล์จริงเป็น string ธรรมดา ไม่ใช่ ISO-8601 timestamp แบบที่ 02-architecture.md ใช้เป็นตัวอย่าง (`2026-06-18T09:00:00Z`) — session ทำหน้าที่เป็น routing key สำหรับ `maw hey` และ log namespacing แต่ไฟล์จริงเลือกใช้ session name ที่คงที่แทน timestamp ที่เปลี่ยนทุกวัน สามฟิลด์นี้ดูเหมือนแค่ metadata แต่หายฟิลด์ไหนไป ทีมไม่มีวันขึ้น

2.2 engine resolution — charter engines block ชนะ config

เสมอ

Resolution chain ตามภาค 1 มีสี่ขั้น เริ่มจาก member's `engine` field ในไฟล์ ไปที่ `engines` block ของ charter ไปที่ `maw config commands` แล้วค่อย fallback แบบ hard-coded — v26.6.14 เคยพึ่งตรง lookup ของ engines block เอง alias สั้นๆ อย่าง `fast` ถูกส่งตรงเข้า API เป็น model identifier แทนที่จะ resolve ผ่าน block ก่อน ทีมดูเหมือนขึ้นสำเร็จ (tmux window เปิด process start) แต่ coder error แล้วออกทันที

charter ของ codex-fanout เลี่ยงปัญหานี้ทั้งหมดด้วยการไม่ใช้ alias สั้นเลย engines block มีแค่ตัวเดียวคือ `omx-5` ที่ผูกกับ shell command เต็มรูปแบบ — ไม่ใช่แค่ model name แต่เป็นทั้ง bootstrap chain: เรียก `codex-setup.ts` ก่อน ตั้ง

`CODEX_HOME` ตั้ง `OMX_AUTO_UPDATE=0` แล้วค่อย exec `omx --direct --madmax` เข้าไป

field `engine: omx-5` ในสมาชิก `codex-1` ก็ชี้ตรงไปที่ key นี้ ไม่มีชั้น alias คั่นกลางให้พังซ้ำ

ส่วน lead ใช้ `engine: claude` ตรงๆ ไม่ผ่าน engines block เลย เพราะ `claude` เป็น engine ที่ maw รู้จักอยู่แล้วในระดับ config global สองแนวทางนี้อยู่ในไฟล์เดียวกันได้ — coder ใช้ engines block เพราะต้อง bootstrap ชับซ้อน ส่วน lead ใช้ fully-qualified name ตรงตามคำแนะนำของภาค 1 ที่ให้เลี่ยง alias lookup เมื่อไม่แน่ใจเวอร์ชัน maw

2.3 v1 vs v2 contract — defaults.worktree หายไปยังไง

pattern จากภาค 1 ไม่มี top-level `lifecycle` block เลย — charter ตัวอย่างจบที่ `members` list สมาชิกแต่ละคนมี `branch` field เดี่ยวๆ ไม่มี field ชื่อ `worktree` แยกออกมาต่างหาก นั่นคือ contract แบบ v1 ที่สมมติว่าทุก member ต้องมี worktree เสมอ ไม่ต้องประกาศชัด

charter ของ `codex-fanout` เป็น contract คนละแบบ มี

```
lifecycle: { worktree: true, merge_on_shutdown: false }
```

 อยู่ท้ายไฟล์ และในแต่ละ member ก็มี `worktree` field ของตัวเองแยกจาก `branch` — v2 contract นี้ไม่มี

```
defaults.worktree
```

 เป็น block กลางให้ inherit ทุก member ต้องประกาศ `worktree` เองตรงๆ ในระดับ member ไม่ใช่ปล่อยให้ lifecycle block เป็นตัวกำหนด default แทน ตรงนี้แหละที่คอมเมนต์บนสุดของไฟล์เตือนไว้เป็นพิเศษ — bug maw-rs #658 ทำให้

```
worktree: / branch: path
```

 ของสมาชิกคนสุดท้ายในไฟล์โดนตัด prefix `agents/` ออกตอน preflight กับ team up (canonicalize fail) เพราะ `codex-1` เป็นคนสุดท้ายในไฟล์นี้ ทีมงานเลยต้องสร้าง symlink `codex-1 → agents/codex-1` ไว้ที่ root ของ repo เป็นทางแก้ชั่วคราว จนกว่า #658 จะ ship ถ้าไม่อยากพึ่ง symlink ก็แค่ย้าย lead ให้เป็นสมาชิกคนสุดท้ายแทน — proof ตรงๆ ว่า v2 contract ยังมี edge case ที่ v1 ไม่เคยเจอ เพราะ v1 ไม่มี per-member worktree path ให้ bug แบบนี้เกิดได้ตั้งแต่ต้น

2.4 lead vs coder — `worktree:false` คือสิ่งที่แยกสองบทบาท

field เดียวที่ทำให้ lead กับ coder เป็นคนละบทบาทกันจริงๆ ในไฟล์นี้ไม่ใช่ `role` แต่เป็น `worktree` — lead มี `worktree: false` ส่วน codex-1 มี

`worktree: agents/codex-1` เป็น path จริง

`role: lead` เขียนไว้ก็จริง แต่ `role` แคบอก label สำหรับ prompt กับ log ไม่ได้ผูกกับ behavior อัตโนมัติ behavior จริงที่แยก lead ออกจาก coder คือ `worktree: false`

— lead ทำงานอยู่บน checkout หลักของ repo ตรงๆ ไม่มี worktree แยก ส่วน branch ของ lead คือ `alpha` ซึ่งเป็นเป้าหมายของทุก PR ตาม goal ที่เขียนไว้บนสุด (`PR → alpha only, never main`)

ทำไมต้องแยกแบบนี้ เพราะ lead มีหน้าที่ merge — prompt ของ lead เขียนชัดว่า “Lead orchestrator. NEVER write code yourself” และ “Merge is the lead’s job” ถ้า lead มี worktree แยกของตัวเอง การ merge PR จาก coder เข้า alpha จะต้องสลับ context ไปมาระหว่าง worktree สองที่ ในขณะที่ coder ต้องแยก worktree เพราะต้อง “implement MINIMAL precise code” โดยไม่ชนกับใคร — ตรงตาม worktree isolation principle จากภาค 1 ที่บอกว่าหน่วยแยก parallel work ที่แท้จริงคือ git worktree ไม่ใช่ process

`worktree: false` จึงไม่ใช่แค่ optional flag ที่ปิดไว้เฉยๆ แต่เป็นตัวกำหนด role ทางสถาปัตยกรรมจริงๆ lead คือคนเดียวที่แตะ checkout หลัก ส่วนใครมี worktree เป็น path จริง คนนั้นคือ coder ที่ต้องทำงานแยกตัว ห้ามแตะ checkout ของ lead หรือ worktree ของคนอื่นเด็ดขาด ตามที่ prompt ของ codex-1 กำกับไว้ตรงๆ ว่า “Never touch lead’s checkout or other worktrees”

Charter หนึ่งไฟล์ผ่าตัดออกมาแล้วเห็นชัดว่าไม่มี field ไหน “แค่ metadata” จริงๆ เลย — `name` ผิดที่ก็หาทีมไม่เจอ `project` เขียน relative ผิดจุดก็ resolve worktree พลาด

`engines` alias ผิดขั้นก็ตายเงียบตั้งแต่ boot และ `worktree` ที่ดูเหมือน flag เล็กๆ กลับเป็นตัวตัดสินว่าใครทำหน้าที่อะไรในทีม

v2 contract ที่ตัด `defaults.worktree` ออกไปแลกมาด้วยความชัดเจนต่อ member แต่ก็แลกมาด้วย bug อย่าง #658 ที่ยังไม่ ship fix เต็มรูปแบบ — นี่คือนโยบายที่ต้องจ่ายเมื่อ contract เปลี่ยนรุ่นเร็วกว่า tooling จะตามทัน

แต่ charter ที่ถูกต้องทุก field ยังไม่พอจะทำให้ทีมขึ้นมาได้จริง — ไฟล์บนดิสก์เป็นแค่พิมพ์เขียว การเดินทางจาก `maw team preflight` ไปจนถึง `maw team up` ที่ใช้งานได้จริงต้องผ่าน symlink workaround, canonicalize fail, และ boot pitfall อีกหลายจุดที่

02-architecture.md เขียนเป็นทฤษฎีไว้ แต่ session จริงของ codex-fanout เจอมาคนละแบบ — บทที่ 3 จะตามรอยการเดินทางนั้นทีละก้าว

บทที่ 3: การเดินทางของ Setup

repo วางเปล่าไม่ได้แปลว่าพร้อม — ประโยคนี้ฟังดูเหมือนเรื่องเบสิกที่ไม่น่าต้องพูด แต่พอ codex-fanout session เจอเข้ากับตัวเอง ถึงรู้ว่ามันคือบล็อกเกอร์จริงที่รอทุกทีมอยู่ ก่อนจะมี coder สักตัวเขียนโค้ดได้ ต้องมี worktree ก่อน ก่อนจะมี worktree ต้องมี branch บน remote ก่อน ก่อนจะมี branch บน remote ต้องมี commit สักตัวหนึ่ง — chain ทั้งหมดนี้ฟังได้ตั้งแต่ข้อแรกสุด และมันมักจะฟังเจียๆ ด้วย

บทนี้เดินทางตาม 4 หัวข้อ เริ่มจาก Trust Prerequisite ที่ทำให้ coder ทำงานได้จริงไม่ใช่แค่ดูเหมือนทำงาน ต่อด้วย 5-Phase Protocol ที่กลั่นจาก 22 ขั้นตอนภาคสนามของ Ting เหลือแค่ 5 gate ที่ต้องผ่านตามลำดับ จากนั้นลงลึกที่บล็อกเกอร์ตัวจริงจาก session 2026-07-23 — repo ที่ไม่มี commit สักบรรทัดเดียว พร้อม error message ตัวเป็นๆ ที่ยืนยันปัญหา ปิดท้ายด้วยเรื่อง pool/account contention ที่เกิดตอนสองทีมใช้บัญชีเดียวกันพร้อมกัน

ภาค 1 สอนว่า trust config คือ prerequisite ที่มองไม่เห็นจนกว่าจะพัง บทนี้เพิ่มอีกชั้นหนึ่ง — repo เองก็เป็น prerequisite ที่มองไม่เห็นเหมือนกัน มันดูมีอยู่แล้ว (`git status` รันได้ ไฟล์อยู่ครบ) แต่ถ้าไม่มี commit สักตัว worktree ก็ผูกถึงไม่ได้ ทุกอย่าง تبدوเหมือนพร้อมกลับกลายเป็นภาพลวงตา

3.1 Trust Prerequisite — ทำไม codex ค้างที่ prompt ถ้าไม่ pre-seed

`CODEX_HOME` ทุกตัวต้องมี `config.toml` พร้อม `trust_level = "trusted"` ก่อนจะสั่ง spawn อะไรทั้งนั้น ไม่ใช่ default ไม่ใช่ optional เพราะถ้าไม่มี codex จะรันในโหมด

sandbox — อ่านไฟล์ได้ วิเคราะห์ได้ แต่เขียนไม่ได้ง่ายๆ coder ที่ขาด trust จะดูเหมือน active ใช้ token ไปเรื่อยๆ แต่ผลลัพธ์คือศูนย์

crew-master-oracle เจอเรื่องนี้ตอน Round 3 Swarm — สอง coder เลือกไฟล์เดียวกันโดยไม่เขียนอะไรลง disk เลยสัปดาห์ ต้นตอไม่ใช่การชนกันของ task แต่เป็น

`~/codex-team/1` ที่ถูกรีเซ็ตระหว่าง cleanup โดยไม่มีใครสังเกต coder ที่ไม่มี trust จะเขียนอะไรไม่ได้และไม่บอกด้วยว่ามีอะไรผิด ต้อง verify ก่อนเสมอ

```
grep trust_level "$CODEX_HOME/config.toml"
# ต้องได้: trust_level = "trusted"
```

ฝั่ง codex-fanout session ไปไกลกว่านั้นอีกขั้น — pre-seed trust เข้าไปตรงๆ ก่อน spawn เลย แทนที่จะรอเช็คที่หลังแล้วค่อยแก้:

```
printf '\n[projects."%s"]\ntrust_level="trusted"\n' "$PWD" >> .codex/
config.toml
```

ขั้นตอนนี้อาจมาจากคำแนะนำของ maw-rs ตอนถูกถามสดๆ ว่า “gotcha สำหรับ fresh worktree มีอะไรบ้าง” — คำตอบคือ pre-seed ก่อนเลย อย่างรอให้ preflight ไปเจอเอง เพราะถ้าเจอตอนนั้นแปลว่า worktree ตัวนั้นยังไม่พร้อม ต้อง full stop

3.2 5-Phase Protocol (repo prep → verify → spawn one → probe → scale)

Ting เคยมี checklist 22 ขั้นตอนกระจายอยู่ 4 หน้า field notes จาก 7 coder กว่า 20 PR โครงสร้างที่กลั่นออกมาเหลือ 5 phase — Repo Prep → Verify → Spawn One → Probe → Scale — แต่ละ phase มี go/no-go gate ของตัวเอง ถ้า gate ไหนไม่ผ่าน ห้ามเดินต่อ

Phase	ชื่อ	เงื่อนไขผ่าน
1	Repo Prep	มี remote branch, worktree สะอาด
2	Verify	<code>preflight.sh</code> exit 0, worktree list ตรงตามที่คาด
3	Spawn One	coder ตัวเดียวรันอยู่, <code>maw peek</code> เห็นว่า active
4	Probe	loop เต็มพิสูจน์แล้ว: <code>dispatch</code> → <code>code</code> → <code>push</code> → <code>PR</code> → <code>merge</code>
5	Scale	coder ที่เหลือ spawn บน task ที่ไม่ทับกัน

ห้ามรัน Phase 5 ก่อน Phase 4 เสร็จ — นี่คือการผิดพลาดที่พบบ่อยที่สุดใน multi-coder deployment ทีมที่รีบ spawn ทั้งสามตัวพร้อมกันมักจะไปชนกันที่ role เดียวกัน ตอน Phase 4 (อย่างกรณี `QUICK-START.md` ถูกมอบหมายให้สองคน) แล้วต้องเสียเวลา 20 นาทีคือ merge ที่ไม่มีวันเกิดถ้า loop แรกถูกพิสูจน์ก่อน session codex-fanout เดินตาม logic เดียวกันนี้ แม้จะไม่ได้เรียกชื่อ phase ตรงๆ ก็ตาม — ขั้นตอนจาก maw-rs ระบุว่า “SPAWN ONE FIRST (the golden rule)” พร้อมลำดับ: `git worktree add` → `cd` → `setup script` → `pre-seed trust` → `maw team load --no-spawn` → `maw team up --only <role>` แล้วค่อย verify boot ด้วย `maw peek` ว่าเห็น engine UI จริง ไม่ใช่แค่ shell หรือ trust prompt ค้างอยู่ ก่อนจะขยับไป `dispatch` task จริง

Phase 4 ในเคสนี้คือ probe task ง่ายๆ — สร้าง `NOTES.md`, commit, push, เปิด PR แล้วรายงานกลับ codex-1 ทำครบทุกขั้นตอน แค่ออนรายงานกลับต้นส่งผิดที่ (เดี๋ยวจะเล่าในบทที่ 4) แต่ loop หลักคือ `dispatch` → `code` → `push` → `PR` → `merge` ผ่านครบ พิสูจน์ว่า environment พร้อมก่อนจะไปคิดเรื่อง scale

3.3 บล็อกเกอร์จริง — repo วางเปล่า ไม่มี commit ไม่มี origin

นี่คือจุดที่ทฤษฎีชนกับของจริง — codex-fanout เป็น repo ที่เพิ่งสร้างใหม่ ไม่มี commit สักตัวเดียว พอลอง `git worktree add` บั๊บ พังทันที เพราะ `HEAD` ยัง unborn อยู่ ผูกถึงอะไรไม่ได้เลย


```
$ git log --oneline -1
fatal: your current branch 'main' does not have any commits yet

$ git ls-remote origin
(nothing)
```

error message นี้ตรงตัว — ไม่มี commit แปลว่าไม่มีจุดอ้างอิงให้ worktree แตกกิ่งออกไป และ coder เองก็ต้องมี content บน `origin` ให้ push กลับไปทับ เปิด PR ได้ ถ้า origin ว่างเปล่าเหมือนกัน push ก็ไม่มีเป้าหมาย

ทางแก้ตรงไปตรงมา — สร้าง `README.md` ขึ้นต่ำ commit push ขึ้น `origin main` แล้วสร้างพร้อม push branch `alpha` (เป้าหมายของ PR ตาม convention “PR → alpha only, never main”):

```
git add . && git commit -m "chore: initial scaffold"
git push -u origin main
git checkout -b alpha
git push -u origin alpha
```

Gate 1 ของ Phase Repo Prep คือ `git branch -r | grep alpha` ต้อง exit 0 ตอนนี้ผ่านแล้ว ประเด็นสำคัญคือ empty repo ไม่ได้ error ให้เห็นตอนวางแผน มันจะโผล่มาตอน `git worktree add` พังเท่านั้น — เป็น prerequisite ที่มองไม่เห็นจนกว่าจะพัง ตรงกับ dna ของบทนี้เป๊ะๆ

แล้วทำไมไม่ตรวจตั้งแต่แรกล่ะ? เพราะ repo ที่มีไฟล์อยู่ครบ `git status` รันผ่าน ดูเผินๆ เหมือนพร้อมทุกอย่าง แต่ commit history คือสิ่งที่ต้องเช็คแยกต่างหาก ไม่ใช่แค่ไฟล์บน disk

3.4 pool/account contention — บัญชีเดียวกัน ชิดจำกัดเดียวกัน

ก่อนจะแตะ pool 5 session codex-fanout เช็คก่อนว่าใช้ได้จริงไหม:

```
ls ~/.codex-team/5/auth.json # exists — pool 5 auth ไว้แล้ว
```

pool 5 auth ไว้แล้วก็จริง แต่ปัญหาคือ maw-rs เองก็กำลังรัน 4 coder อยู่บน pool 1/2/5/6 พร้อมกัน — pool 5 เฉพาะเจาะจงถูกใช้โดย coder ชื่อ `infra` ของ maw-rs สถานการณ์นี้ต้องส่งแผนกลับไปให้ maw-rs ดูก่อนแต่อะไรทั้งนั้น เพราะการแชร์บัญชีเดียวกันมีผลกระทบข้ามทีม ไม่ใช่แค่ในทีมตัวเอง

maw-rs ยืนยันว่าใช้ร่วมกันได้ — **shared credential, shared rate limit แต่แต่ละ coder มี worktree-local CODEX_HOME แยกกัน** (SQLite/lock แยกกันคนละชุด)

contention ที่จะเกิดคือ rate-limit ทำให้ช้าลง ไม่ใช่ hard conflict และรับได้สำหรับ coder เบาๆ ตัวเดียว ส่วน pool 3/4/7 ที่ dedicated ไว้กลับยังไม่ auth — ต้อง manual `codex login` ซึ่งอยู่นอก scope ของ “fast” ไปเลย เลยเลือก pool 5 ตามเดิม จุดนี้ต่างจาก trap #2 ในภาค 1 (shared CODEX_HOME ทำให้ SQLite lock ชน จนฟัง 12% ของ agent) ตรงที่ประเด็นในภาค 2 ไม่ใช่ CODEX_HOME ที่แชร์กัน — เพราะแต่ละ coder แยก worktree-local CODEX_HOME กันอยู่แล้ว แต่เป็น**บัญชี** ที่แชร์กัน ผลคือ rate limit ไม่ใช่ lock contention คนละชั้นของปัญหา แม้จะฟังดูคล้ายกันตอนแรก

setup ที่ดูเหมือนงานเตรียมการเล็กๆ กลายเป็นครึ่งหนึ่งของเวลาทั้ง session — จาก consult เย็นชากับ maw-rs ไปจนถึง commit แรกบน `alpha` ก่อนจะมี coder สักตัว ขยับได้จริง 4 หัวข้อในบทนี้ผูกกันเป็นเส้นเดียว trust ต้อง pre-seed, phase ต้องเรียงลำดับ, repo ต้องมี commit, pool ต้องเช็คก่อนแตะ — พลาดจุดไหนจุดหนึ่ง ทั้ง chain ก็สะดุด

บทที่ 4 จะพาไปดูว่าทีมขนาด 1 คนขยายไปเป็นพันได้อย่างไร จาก probe task เดียวของ codex-1 ที่ยิ่งกลับผิดที่ (“no window ‘lead’ in session”) ไปจนถึง 1,000 Haiku agent ที่วิ่งพร้อมกันได้ใน 103 วินาที — เส้นทางจาก Solo ถึง Thousand ที่พิสูจน์ว่า

architecture ที่ถูกต้องตั้งแต่ setup คือสิ่งที่ทำให้ scale ได้จริง ไม่ใช่แค่ workaround เฉพาะหน้า

บทที่ 4: จาก Solo ถึง Thousand — สเกลที่พิสูจน์แล้ว

ทีมของ codex-fanout ที่เล่าในภาค 2 ไม่ได้เริ่มจากศูนย์ มันยืนอยู่บนบทเรียนที่ crew-master-oracle พิสูจน์มาก่อนแล้วห้ารอบ แต่ละรอบทดสอบสเกลที่ต่างกัน — solo หนึ่งตัว, trio สามตัว, swarm สามตัวไม่มีผู้นำ, tournament สามตัวแข่งกัน, และ thousand หนึ่งพันตัวพร้อมกัน บทนี้ไม่ได้เล่าซ้ำทุกตัวเลข แต่เลือกเฉพาะที่สำคัญที่สุดจากแต่ละรอบ — proven experiment จาก crew-master-oracle ล้วน ไม่มีตัวเลขไหนมาจาก session ปัจจุบันของ codex-fanout

ประเด็นที่ผูกทุกรอบเข้าด้วยกันคือเรื่องเดียว ต้นทุนต่อ coder ไม่ได้ลดลงแบบ linear เมื่อสเกลใหญ่ขึ้น บางครั้งมันดีขึ้น บางครั้งมันแย่ลงแบบคาดไม่ถึง solo ตัวเดียวชนะทีมสองคนได้ เพราะ overhead การ spawn agent ตัวที่สองแพงกว่างานที่ต้องทำเสียอีก แต่พอถึงพันตัว ต้นทุนต่อ agent กลับนิ่งเป็นเส้นตรง — トラバタイงานเบาพอ พอเจองานหนัก เส้นตรงนั้นหักทันที

dna ของบทนี้คือ “each scale breaks differently” bug ที่ solo ไม่มีวันเจอ จะโผล่ตอน swarm เท่านั้น และ bug ที่ swarm เจอ ก็ไม่ใช่ bug เดียวกับที่ thousand เจอ นี่ไม่ใช่เรื่องบังเอิญ — มันคือ physics ของ coordination ที่เปลี่ยนรูปแบบทุกครั้งที่จำนวน agent ข้าม threshold หนึ่งไป ใครที่คิดว่าเข้าใจ multi-agent จากการรัน trio ครั้งเดียวจะพลาดกับดักที่ swarm วางไว้เต็มๆ

4.1 The Solo — null hypothesis ที่ไม่จริงอย่างที่คิด

สมมติฐานของรอบแรกฟังดูดีแย่งง่าย coder ตัวเดียวไม่มี lead จะเร็วกว่า ถูกกว่า และพลาดน้อยกว่าทีมที่ประสานงานกัน — สำหรับงานเล็ก (ต่ำกว่า 50 บรรทัด) ทีม crew-

master-oracle ทดสอบด้วยงานจริง สร้าง `.gitignore` กับ `CLAUDE.md` ให้ agent ตัวเดียวชื่อ `omx-1` ไม่มี lead ไม่มี orchestration layer ผลลัพธ์ชัดกว่าที่คาด wall-clock ทั้งหมด 2 นาที 52 วินาที ใช้ context แค่ 3% ของ window ไฟล์ทั้งสองถูกต้องตั้งแต่ครั้งแรก ไม่มี conflict ไม่มี rewrite แต่สิ่งที่น่าสนใจกว่าตัวเลขคือพฤติกรรมที่ไม่มีใครสั่ง — สามวินาทีหลังรับงาน `omx-1` สร้าง checklist วางแผนของตัวเองโดยไม่มีคำสั่งให้ทำแบบนั้น นี่คือ emergent self-organization ที่ท้าทายสมมติฐานพื้นฐานของ lead-coder topology เลย ถ้า coder วางแผนเองได้เมื่องานอยู่ในขอบเขตที่เหมาะสม หน้าที่วางแผนของ lead ก็ซ้ำซ้อนสำหรับงานระดับนี้ ทีมงานประเมินไว้ด้วยว่าถ้าใช้ lead + coder แทน solo งานเดียวกันนี้จะกินเวลาประมาณ 5 นาทีครึ่ง แพงกว่าเกือบเท่าตัว ผลลัพธ์เหมือนกันเป๊ะ — solo จึงไม่ใช่ทางเลือกสำรอง มันคือ topology ที่เหมาะสมที่สุดสำหรับงานประเภทหนึ่ง เงื่อนไขการทำลายกฎ 50 บรรทัดมีอยู่จริงเหมือนกัน เมื่อไฟล์ต้องเขียนพร้อมกัน เมื่องานต้องการความเชี่ยวชาญต่างสาย เมื่อ deadline บังคับให้ทำนานกัน หรือเมื่อต้องมีคนตรวจสอบอิสระจากคนทำ — สี่เงื่อนไขนี้ไม่มีข้อไหนอยู่ในรอบแรกเลยสักข้อ

4.2 The Trio — token economics เริ่มมีความหมาย

รอบสองขยับไปสามตัว lead หนึ่ง coder สาม แต่ละตัวได้ไฟล์ของตัวเอง ไม่แตะไฟล์กัน สมมติฐานคืองานที่แยกกันได้จริง (file-disjoint) จะเร็วกว่าทำทีละตัวตามลำดับ ผลออกมาคือ 1.8 เท่า ไม่ใช่ 3 เท่าตามทฤษฎี และช่องว่างระหว่าง 1.8 กับ 3.0 นั้นคือบทเรียนทางวิศวกรรมล้วนๆ ตัวถ่วงหลักคือ spawn overhead เกือบสองนาทีก่อนเริ่มเขียนโค้ดสักบรรทัด บวกกับ agent ที่ทำงานเร็วสุดต้องนั่งรอ agent ที่ช้าสุด — `omx-3` เสร็จงานแล้วนั่งเฉยไป 1 นาที 43 วินาที เพราะ lead แจกงานแบบ fixed list ไม่มี dynamic dispatch มารับงานต่อ ที่สำคัญกว่านั้นคือเรื่อง token ต่อบรรทัด parallelism ไม่ได้ลด token ต่องาน มันแค่ลดเวลา รอบนี้ใช้ token รวมประมาณ 694,000 สำหรับโค้ด 333 บรรทัด แต่ token/line ไม่เท่ากันในแต่ละ agent — งานที่มี conditional logic ซ้ำซ้อนกิน token ต่อบรรทัดสูงกว่า

งานที่เป็น linear cleanup token-per-line ไม่ใช่ตัวชี้วัดคุณภาพ มันคือสัญญาณความลึกของการให้เหตุผล

```
# Token cost model (Sonnet 4.x, ~70/30 input/output split)
tokens_total = 694_000
cost_input = (tokens_total * 0.70 / 1_000_000) * 3.00 # $1.457
cost_output = (tokens_total * 0.30 / 1_000_000) * 15.00 # $3.123
total_cost = cost_input + cost_output # ~$4.58

# Break-even for parallel formation:
# Parallel pays off when S > P + O
# S = serial total, P = slowest parallel task, O = formation overhead
# ในรอบนี้: 11m36s > 4m53s + 1m52s = 6m45s ✓
```

trio คือ formation ขนาดเล็กที่สุดที่ยังคุ้มค่าจะรัน มันเล็กพอจะ debug ได้เมื่อพัง และใหญ่พอจะเห็น speedup principle ชัดเจน สิ่ง trio สอนไว้ — ทั้ง bottleneck ของ slowest worker, idle capacity ที่สูญเปล่า, ความเสี่ยงจาก semantic drift ระหว่างไฟล์ — ทุก formation ที่ใหญ่กว่าจะสืบทอด failure mode พวกนี้ในความเข้มข้นที่สูงขึ้น

4.3 The Swarm — เมื่อ isolated worktree ยังไม่พอ

รอบสามคือรอบที่พังชดที่สุดในห้ารอบ สาม coder ได้รับคำสั่งเดียวกันเบ๊ “เลือกงานหนึ่งที่ยังไม่มีใครจับ” ไม่มีกลไกประสานงานใดๆ ผลคือ codex-1 กับ codex-3 เลือกงานเดียวกัน (Task 3 ที่ซื้อสื่อความชัดที่สุด) ส่วน Task 2 ไม่มีใครแตะเลย collision rate 66.7% — parallelism ที่ effective แค่ 0.67 เท่า แย่กว่าใช้ agent ตัวเดียวทำตามลำดับเสียอีก worktree isolation ที่ป้องกัน merge conflict ได้ดี กลับเป็นตัวปิดกั้นการรับรู้ซึ่งกันและกันไปด้วยในตัว agent แต่ละตัวไม่เห็น working state ของอีกตัว ไม่มี shared state ไม่มี lock ไม่มี linearization point — นี่คือ classic thundering herd ผสมกับ check-then-act race condition ที่ทฤษฎี distributed systems อธิบายไว้ตั้งแต่ยุค Brooks (1975) และ Lamport (1978) แล้ว

ที่ทำให้เรื่องซับซ้อนขึ้นคือการทดสอบสเกล 1,000 agent คู่ขนานกันในวันเดียวกัน ได้ผล 100% สำเร็จ ไม่มี collision เลย ฟังดูขัดแย้งกับรอบสาม แต่จริงๆ ไม่ใช่ — การทดสอบพันตัวนั้นไม่มี shared resource เลย แต่ละ agent แคพูด “hello” โดยไม่แตะทรัพยากรร่วมกับใคร collision probability เป็นฟังก์ชันของ resource scarcity ไม่ใช่จำนวน agent ที่ scarcity เป็นศูนย์ collision ก็เป็นศูนย์เสมอ ไม่ว่าจะกี่ agent ก็ตัวก็ตามทางแก้มันซับซ้อนเลย ไฟล์ lock แบบ `o_excl`, GitHub issue assignment, หรือ coordinator process ตัวเดียว — อย่างใดอย่างหนึ่งพอจะทำลาย thundering herd ได้รอบที่สี่เอา GitHub issue assignment มาใส่ก่อน dispatch collision rate ตกลงจาก 67% เหลือ 0% ทันที ใช้เวลาเพิ่มแค่ 3 วินาที แลกกับการประหยัด compute ที่เสียไปทั้งรอบ

4.4 The Tournament และ The Thousand — token

efficiency inversion + throughput formula

รอบสี่เปลี่ยนคำถามจาก “จะประสานงานยังไงไม่ให้ชน” เป็น “ถ้าให้สาม coder ทำงานเดียวกันพร้อมกันแล้วเลือกที่ดีที่สุด จะคุ้มไหม” กลไกไม่ต่างจาก swarm เลยสักนิด dispatch message เหมือนกันทุกตัว ต่างกันแค่ขั้นตอนสุดท้าย — swarm synthesize ผลรวม ส่วน tournament เลือกผู้ชนะ

ผลที่พลิกความคาดหมายที่สุดคือเรื่อง token codex-1 ใช้ token มากกว่า codex-2 ถึง 2.2 เท่า (1.7 ล้าน เทียบ 766,000) แต่ได้บรรทัดโค้ดน้อยกว่าและเข้าเส้นชัยเป็นอันดับสุดท้าย token ที่มากขึ้นไม่ได้แปลว่าคุณภาพดีขึ้น มันมักแปลว่า agent ใช้เวลาไปกับการ deliberate ในชั้นวางแผนมากเกินไป จนเสียเปรียบ agent ที่ commit เร็วกว่าและลงมือ implement ตรงกว่า — token efficiency inversion นี่คือเหตุผลว่าทำไมงานที่ต้องการความเร็วและ coverage ควรมอบให้ agent ที่ context เบา ส่วนงานที่ต้องการ coherence ลึกค่อยเก็บ high-context agent ไว้ใช้

รอบห้าคือรอบที่ใหญ่ที่สุด หนึ่งพัน agent พร้อมกัน ทั้งหมดสำเร็จ 100% ใน 103.3 วินาที เฉลี่ย 9.7 agent ต่อวินาที ตัวเลขนี้ไม่ได้มาจากการยิง 1,000 คำขอพร้อมกันทีเดียว — มันมาจากสูตรง่ายๆ ที่ทีมงานเรียกว่าสูตรเดียวที่ต้องจำ

```
# Throughput formula – engrave this
def agents_per_second(concurrent_cap: int, avg_task_duration_s: float)
→ float:
    return concurrent_cap / avg_task_duration_s

agents_per_second(16, 1.65)    # hello-world Haiku    → 9.7 agents/s
agents_per_second(16, 8.0)     # summarize document → 2.0 agents/s
agents_per_second(16, 45.0)    # codex coder, boot    → 0.36 agents/s
agents_per_second(16, 75.0)    # heavy tmux coder     → 0.21 agents/s
```

concurrent cap ไม่ใช่ตัวกำหนด throughput สูงสุดอย่างที่เราคุ้นเคย มันแค่กำหนดจำนวน slot ที่ทำงานพร้อมกัน ณ ขณะหนึ่ง ตัวที่กำหนด throughput จริงคือ task duration เฉลี่ย งานเบาเท่าไรก็ยิ่งดันผ่าน cap ได้มากเท่านั้น นี่คือเหตุผลที่การทดสอบใช้ Haiku ล้วน ไม่ใช่เพราะ Haiku ฉลาดพอ แต่เพราะต้องการทดสอบ scaffolding ไม่ใช่ทดสอบโมเดล

แต่ตัวเลขนี้ใช้ไม่ได้กับงานหนัก tmux codex coder ตัวจริงที่ boot session, authenticate, โหลด context จากโค้ดจริง, reasoning หลายขั้นตอน ใช้เวลาเฉลี่ย 75 วินาทีต่อ task เทียบกับ 1.65 วินาทีของ hello-world เท่ากัน 20-80 เท่า ถ้าจะรันพัน coder หนักขนาดนี้ผ่าน cap เดียวกัน จะใช้เวลาประมาณ 79 นาที ไม่ใช่ 103 วินาที และก่อนจะถึง cap 16 slot ด้วยซ้ำ RAM ก็เต็มก่อนแล้ว — 15 tmux session กินไปเกือบ 14GB บนเครื่อง 16GB การทดสอบจริงจึงใช้ 5 ทีม 3 coder ต่อทีม กระจาย API key แยกต่อทีม เพื่อไม่ให้ rate limit ของทีมหนึ่งไปล็อกทีมอื่น

บทเรียนที่ทั้งห้ารอบทิ้งไว้ให้ร่วมกันคือ orchestration layer นั้น scale ได้เป็นเส้นตรงจริง แต่ hardware ไม่เคยเป็นเส้นตรง เพดานที่แท้จริงไม่ได้อยู่ที่ทฤษฎี multi-agent มันอยู่ที่ CPU, RAM, และ rate limit ของ API เสมอ

หำรอบนี้คือรากฐานที่ codex-fanout ยืนอยู่ก่อนจะเจอสถานการณ์จริงของตัวเอง solo บอกว่าเมื่อไหร่ไม่ต้องใช้ทีม trio บอกวิธีคิด break-even ก่อน spawn swarm เตือนว่า isolation ไม่เท่ากับ coordination tournament เผยว่า token มากไม่ได้แปลว่าดีกว่า และ thousand ให้สูตรคำนวณ throughput ที่ใช้ได้จริง แต่ตัวเลขทั้งหมดนี้มาจากสภาพแวดล้อมที่ควบคุมได้ — งานที่กำหนดไว้ล่วงหน้า ไม่มี merge conflict ที่ซับซ้อน ไม่มี agent ที่ดื้อคำสั่ง

บทที่ 5 จะพาไปสู่ด้านที่ไม่ได้ถูกทดสอบในหำรอบนี้ — The Lock และ Twelve Traps เมื่อ 16 agent ต้องแย่งทรัพยากรเดียวกันจริงๆ และไม่มีใครยอมถอย จะเกิดอะไรขึ้นเมื่อ coordination protocol ที่ดูดีในกระดาษ ต้องเจอกับ agent ที่ตีความคำสั่งคนละแบบ และกับดักสิบสองข้อที่รอทีมทุกทีมที่คิดว่าตัวเองเตรียมพร้อมแล้ว

บทที่ 5: The Lock และ Twelve Traps

บักที่แพงที่สุดในระบบ distributed ไม่ใช่บักที่ error ดัง ๆ แล้วล่มทั้งระบบ — บักที่แพงที่สุดคือบักที่ทำงาน “สำเร็จ” แต่สำเร็จผิดอย่าง เจียบสนิท ไม่มี log ไม่มี exception ไม่มีใครรู้จนกว่าจะสาย

เรื่องในบทนี้มาจาก session จริงของ crew-master-oracle ที่ codex-1 ตายซ้ำ ๆ ไม่ใช่ตายแบบ crash แต่ตายแบบ boot ขึ้นมาเป็น shell เปล่า ไม่มี task ไม่มี memory ไม่มี goal ระบบรายงานว่า fleet เขียว แต่ coder ไม่ได้ coding อะไรเลย นี่คือ symptom ที่อันตรายกว่า error เพราะไม่มีสัญญาณให้จับ

การไล่ root cause ใช้เวลาสามสมมติฐาน สองข้อแรกผิด ข้อที่สามถูก — และวิธีที่ทีมพิสูจน์แต่ละข้อคือบทเรียนที่สำคัญพอ ๆ กับตัวบักเอง ต่อจากนั้นบทนี้จะสรุป Twelve Traps ที่ทีมเดียวกันเจอตลอด session และปิดท้ายด้วยการเปรียบเทียบกับบัก #658 ที่เจอเองในภาค 2 ของหนังสือเล่มนี้ — pattern เดียวกัน คนละบริบท

5.1 อาการที่ไม่มี error

codex-1 บูดซ้ำแล้วซ้ำเล่า ทุกครั้งจบที่ shell เปล่า ไม่มี context ไม่มีอะไรให้ agent ทำงานต่อ จากภายนอกดูเหมือน initialization ค้างกลางทาง แต่ process ยังรันอยู่ — เทคนิคแล้วคือ “alive” แต่ใช้งานจริงไม่ได้เลย

fleet monitor รายงานสถานะปกติทุกอย่าง เขียวหมด แต่ coder ไม่ผลิตอะไรออกมา ระบบที่ดูสุขภาพดีในขณะที่ไม่ผลิตผลลัพธ์ ยากกว่าระบบที่ล่มชัดเจนเยอะ เพราะไม่มี stack trace ให้ตาม ไม่มี exit code ให้ grep

ทีมตั้งสมมติฐานสามข้อเรียงกัน แต่ละข้อฟังดูสมเหตุสมผล แต่ละข้อพิสูจน์ไม่ตรง มีแค่ข้อที่สามที่ยืนยันได้ด้วยเครื่องมือระดับ process — `lsuf` กับ `ps eww`

5.2 กลไก และวิวัฒนาการของ fix

สมมติฐานแรกโทษ `reasoning_effort=low` — คิดว่า reasoning ทำให้ agent parse task state ไม่ออก แล้วก็ fallback ไป shell ปิด flag นี้แล้วอาการไม่หาย พิสูจน์แล้วว่าไม่ใช่จุดนี้

สมมติฐานที่สองโทษ token expiry — คิดว่า credential หมดอายุทำให้ auth ล้มเหลว แล้ว process ตกไป shell แทนที่จะ error ตรง ๆ แต่มี screenshot คัดค้านตรง ๆ: `omx`

ใช้ credential pool เดียวกัน บุตปกติ ถ้า token หมดจริง `omx` บุตไม่ได้แน่ —

screenshot นี้คือ falsifier ที่ปฏิเสธไม่ได้ บทเรียนตรงนี้ไม่ใช่ “เก็บข้อมูลให้มากก่อนตั้งสมมติฐาน” แต่คือหาทางพิสูจน์ว่าสมมติฐานตัวเองผิดทันทีที่ตั้งขึ้นมา

สมมติฐานที่สามต้องใช้เครื่องมือระดับ process จริง ๆ ทีมส่ง 5-agent workflow ไปรัน

`ls -lsof` ดู open file handle กับ `ps -eww` ดู environment variable ทั้ง fleet ผลออกมาชัดเจน

```
$ ps eww | grep CODEX_HOME | grep -o 'CODEX_HOME=[^ ]*' | sort | uniq -c | sort -rn
17 CODEX_HOME=/Users/nat/.codex-team/1
```

สับสน process ชี้ไปที่ directory เดียวกัน — SQLite lock contention คือ root cause ตัวจริง

กลไกคือ Codex เก็บ state ใน SQLite แล้วล็อกไฟล์ด้วย PID lock ตอน startup พอมี process มากกว่าหนึ่งชี้ไปที่ `CODEX_HOME` เดียวกัน คนแรกก็ถึงล็อกได้ก็ยึดไว้ ที่เหลืออีกสับ

ทุกคนรอจนหมดเวลาแล้วก็บูตต่อโดยไม่มี state — ไม่ error ไม่ crash แคร่รันต่อแบบว่าง

เปล่า `waitForNonShell` เช็คแค่ว่า process ยังมีชีวิตอยู่ ไม่ได้เช็คความมั่นคง state

สำเร็จหรือเปล่า presence ไม่เท่ากับ readiness — process ที่เริ่มแล้วไม่ได้แปลว่า

process ที่พร้อมใช้งาน

fix ผ่านสองรอบ รอบแรกให้แต่ละ oracle มี `CODEX_HOME` แยกกัน แก้ contention

ระหว่าง oracle คนละประเภทได้ แต่สอง instance ของ oracle เดียวกันยังชนกันอยู่ดี

รอบสองถึงจะตรงจุด — ผูก `CODEX_HOME` เข้ากับ worktree ผ่าน `codex-setup.ts` แต่
ละ worktree มี `.codex` ของตัวเอง ส่วน auth ยัง symlink มาจาก pool กลางได้เพราะ
auth เป็น read-only ไม่ต้องล็อก แยกสิ่งที่แชร์ได้โดยธรรมชาติ ออกจากสิ่งที่ต้องแยกโดย
สถาปัตยกรรม — สับสนสองอย่างนี้คือบาปต้นทาง
ทดสอบด้วย `maw-rs` รันห้า coder พร้อมกัน — ห้า SQLite file แยกกัน ล็อกไม่ชนกัน
เลยสักครั้ง บุตรครบ state ครบทุกตัว

5.3 Twelve Traps สรุปย่อ

นอกจาก SQLite lock ที่เป็นพระเอกของบทนี้ session เดียวกันยังเจอกับดักอีกสิบเอ็ดจุด
แต่ละจุดเป็นสิ่งที่ทีมที่ไม่เคยเจอมาก่อนจะเจ็บแน่นอน

Charter engine block — engine ที่ประกาศไว้ตอน spawn ไม่ถูกจดทะเบียนใน
config runtime เลยข้ามขั้น charter ไปเจียบ ๆ coder ที่ออกมาไม่มี task boundary ไม่
มี scope เลย fix คือลงทะเบียน engine ใน `~/maw/config.toml` แล้วเช็คด้วย

`maw engine list` ก่อน spawn ทุกครั้ง

`maw team down --only` ฆ่าทุกคน — flag ตั้งใจให้ terminate เฉพาะบางตัว แต่
parsing บั๊กทำให้ ignore selector ฆ่าทั้ง fleด flag ที่ไม่ทำงานอันตรายกว่า flag ที่ไม่มีอยู่
เพราะมันสร้างความมั่นใจปลอมว่าคำสั่งทำงานตามที่สั่ง

SendMessage เจียบสำหรับ omx — omx ไม่ได้ poll inbox ที่ `SendMessage` เขียนไป
ข้อความหายไปเฉย ๆ โดยไม่มี ack ต้องใช้ `maw hey` แทนเสมอ

False negative warning — monitor เตือนว่า “may not have submitted” ทั้งที่
coder commit สำเร็จแล้วแต่ยังไม่ push ทำให้คนแทรกแซง coder ที่กำลังทำงานอยู่ กฎ
คือ peek ก่อนเชื่อ warning

Auto-explore ใน `--madmax` — coder ที่ยังไม่มี task เริ่ม explore repo เองก่อนได้
รับ dispatch เพาโทเคนฟรี ต้องใส่ WAIT directive ใน charter prompt

Generic **codex** ชน Claude Code — เรียก **codex** แบบไม่ระบุ path ชัดเจน ระบบไปเจอ **.claude/** แล้ว resolve ผิดไปที่ Claude Code แทน ต้องระบุชื่อ engine แบบเจาะจง เช่น **omx-3** ไม่ใช่ **codex** เนย ๆ

.codex/ .claude บล็อก **worktree remove** — สองโฟลเดอร์นี้เป็น untracked โดยดีไซน์ **git worktree remove** เลยปฏิเสธลบถ้าไม่ force ทีม maw-rs วัดได้ fail rate ถึง 45% ตอน shutdown ทางแก้คือ **mv** ออกก่อนแล้วค่อย remove

Swarm collision ที่ 67% — dispatch โดยไม่มี atomic task-claiming ทำให้ coder หลายตัวเห็น task เดียวกันแล้วเริ่มทำพร้อมกัน วัดได้ collision rate ราว 67% ทางแก้คือ

UPDATE ... WHERE claimed_by IS NULL แบบ test-and-set

Stale worktree บล็อก **spawn** — worktree เก่าที่ลบไม่หมดชนกับ path ใหม่ ต้อง treat cleanup เป็น idempotent startup logic ไม่ใช่แค่ teardown logic

reasoning_effort=low ผลิตขยะในงาน **planning** — ใช้ได้ดีกับงาน classify/route แต่ในงาน multi-step planning มันตัด reasoning chain สั้นเกินไป ได้ output ที่ syntax ผ่านแต่ semantic พัง

Token spend ไม่แปรผันตรงกับคุณภาพ — coder ที่ใช้โทเคนเยอะสุดในกลุ่ม (1.7M) กลับส่งงานช้าสุดและ revise เยอะสุด token volume เป็น cost metric ไม่ใช่ quality metric

วินิจฉัย “token expired” โดยไม่ peek ก่อน — ความเจียบของ coder ไม่เท่ากับ token หมดอายุเสมอไป อาจแค่กำลังรัน I/O หนกอยู่ การ restart โดยไม่ peek ทำให้เสียงาน checklist ที่ทำไปแล้วหลายสิบนาที สามข้อสรุปที่ลอยขึ้นมาจากสิบสองกับดักนี้คือ — ความเจียบไม่ใช่ความล้มเหลว ต้อง peek ก่อนสรุปทุกครั้ง shared state ฆ่า parallelism ทุกครั้งที่ resource แชนกันโดยไม่ atomic และ default ที่ออกแบบมาสำหรับ agent เดี่ยวอันตรายเมื่อขยายเป็น fleet

5.4 เจาสะท้อนจาก #658

เรื่องทั้งหมดในบทนี้เกิดที่ crew-master-oracle คนละ session คนละทีม แต่ pattern เดียวกันนี้โผล่มาอีกครั้งใน session codex-fanout วันที่ 23 กรกฎาคม 2026 — คราวนี้

ไม่ใช่ SQLite lock แต่เป็นบั๊กใน `maw team preflight` และ `maw team up` ที่ strip prefix `agents/` ออกจาก path ของ member ตัวสุดท้ายใน charter ก่อน canonicalize

อาการหน้าตาคุ้น ๆ — path ที่ควรจะเป็น `agents/codex-1` กลายเป็น `codex-1` เฉย ๆ

ไม่มี error บอกตรง ๆ ว่า “prefix หาย” มีแค่ error ปลายทางที่บอกว่า worktree ไม่พบกับ trust config อ่านไม่ได้ — เหมือนกับ codex-1 ที่บูตเป็น shell เปลา่ตรงที่ทั้งคู่คือ silent failure ที่ต้องไล่ที่ละสมมติฐานกว่าจะเจอต้นตอ

ทีมในบท 7 ก็ทำสิ่งเดียวกับที่บทนี้สอน — ไม่เชื่อสมมติฐานแรก (คิดว่าเป็น stale cwd ก่อน) รันซ้ำจาก clean state เพื่อ falsify แล้วพอยังไม่หาย ก็ escalate ไปหา peer ที่เขียนเครื่องมือตัวนั่นเอง จน reproduce ได้ live แล้วยืนยันเป็นบั๊กจริง — วิธีนี้ตรงกับสิ่งที่บทนี้ย้ำไว้ตลอด: อย่าเชื่อสมมติฐานแรก และเมื่อ instrumentation ในมือไม่พอ ให้ไปหาคนที่

ภาค 1 ของหนังสือเล่มนี้จบตรงนี้ — ห้าบทที่ผ่านมาคือบทเรียนที่พิสูจน์แล้วจาก crew-master-oracle การกำหนด boundary ของทีม การแบ่ง state ที่ต้องแยกจาก state ที่แชร์ได้ และวิธีอ่านความเจ็บไม่ให้อ่านใจผิด ทั้งหมดนี้ไม่ใช่ทฤษฎี แต่เป็นแผนที่เขียนจากรอยแผลจริง

ภาค 2 ตั้งแต่บทที่ 6 เป็นต้นไป จะพาไปดู session codex-fanout ตัวจริง — ทีมที่ประกอบขึ้นจาก lead หนึ่งตัวกับ coder หลายตัว เจอบั๊กใหม่ที่ไม่มีในสิบสองกับดักนี้ ต้อง cold-consult กับ peer เจ้าของเครื่องมือ และต้องตัดสินใจกลางทางว่าจะ workaround แบบไหนถึงจะไม่เสียเวลาไปมากกว่าที่ควร บทที่ 6 จะเริ่มจากการก่อร่างทีมตั้งแต่ศูนย์ ก่อนที่บทที่ 7 จะพาไปดู #658 แบบเต็ม ๆ ว่าทีมไล่ตามรอยยังงั้นจนเจอ fix จริง

บทที่ 6: การถามผู้เชี่ยวชาญแบบเย็น

ภาค 1 ปิดท้ายด้วยการสังเคราะห์บทเรียนจาก crew-master-oracle — ทฤษฎีที่กลั่นมาจากทีมจริงหลายทีม จนถึงจุดนี้ผู้อ่านน่าจะรู้แล้วว่า “ควรทำอะไร” แต่บทนี้เปลี่ยนโหมดจากทฤษฎีไปเป็น case study สดๆ ตัวเดียว บันทึกแบบ timestamp-level จาก session จริงชื่อ `codex-fanout` วันที่ 23 กรกฎาคม 2026

Session นี้เริ่มต้นแบบเปล่าเปลือกที่สุดเท่าที่จะเป็นไปได้ — Claude Sonnet ตัวหนึ่ง รัน reasoning effort ระดับ low ไม่มี briefing มาก่อน ไม่มี context เกี่ยวกับ codex team ในหัวเลยแม้แต่หน่อย งานที่ได้รับคือประโยคสั้นๆ จาก Nat: “maw ls to check and talk to maw-rs how to start make codex team to help us here” แล้วตามด้วยการเจาะจงให้แคบลง — “I just want 1 codex and use account 5” โจทย์ตรงนี้คือปัญหาคลาสสิกที่ทุก orchestrator เจอ — ต้อง spawn ทีมโดยไม่รู้ contract ปัจจุบันของระบบ ทางเลือกมีสองทาง หนึ่งคือเปิด guidebook เก่าแล้วเดาไปตามนั้น สองคือถามคนที่เพิ่งทำงานแบบเดียวกันมาสดๆ บทนี้จะเล่าว่าทางที่สองพาไปถึงไหน และทำไมทางแรกถึงเกือบทำให้ทีมพังตั้งแต่ยังไม่ทันเริ่ม

6.1 บริบท — session ว่างเปล่า ต้องการ 1 coder บัญชี 5

ก่อนพิมพ์คำสั่งแรก session `codex-fanout` ไม่มีอะไรเลย repo ว่าง ไม่มี context งานเก่า ไม่มีแม้แต่ commit สักตัว (เรื่องนี้จะกลายเป็นปัญหาจริงในบทที่ 7) สิ่งเดียวที่มีคือคำสั่งของ Nat กับ session name

โจทย์แคบลงเรื่อยๆ จาก “make codex team” กว้างๆ กลายเป็นตัวเลขชัด — coder ตัวเดียว ใช้บัญชี (pool) หมายเลข 5 นี่คือนัยสำคัญที่ทำให้ปัญหาไม่ใหญ่เกินจัดการ เพราะถ้าตั้งเป้าหมาย 5 coder ตั้งแต่แรก ความเสี่ยงจะพุ่งขึ้นตามจำนวน แต่ตัวเดียวหมายความว่า

ว่า ถ้าอะไรพัง cause-space ก็มีแค่หนึ่งจุด — หลักการเดียวกับที่ TING-ORACLE เรียนรู้มาด้วยราคา 45 นาที (รายละเอียดใน 6.4)
คำถามที่เหลืออยู่คือ “แล้วจะเริ่มยังไง” — charter เขียนแบบไหน engine key ต้องตั้งชื่ออะไร มี pitfall อะไรซ่อนอยู่บ้างสำหรับ fresh worktree ตรงนี้แหละที่ session ตัดสินใจไม่ได้ แต่ไปถามคนที่รู้จริง

6.2 maw hey แบบ cold — ไม่มี context ร่วมกันมาก่อน, maw-rs กำลังยุ่งอยู่กับงานอื่น

`maw ls` แสดง session ทั้งหมด 40 ตัวในเฟล็ต ในนั้นมี `33-maw-rs` — 9 pane กำลังรัน 4-coder fanout ของตัวเองอยู่ นี่คือ oracle ที่รู้เรื่อง codex team contract ล่าสุดจริงๆ เพราะเพิ่งสร้างทีม 5-coder ของตัวเองมาไม่กี่วันก่อนหน้านี้
ไม่มีการทักทายอุ่นเครื่อง ไม่มี pre-coordination มาก่อนเลย ข้อความแรกที่ส่งไปคือ

```
maw hey 33-maw-rs "Hi maw-rs - I'm working in codex-fanout and want to
set up a codex coder
team to help here. What's the recommended flow - maw team up, dry-run
first, charter setup?
Any gotchas for a fresh worktree (e.g. omx boot pitfall)? Please advise
briefly."
```

จุดที่ต้องขีดเส้นใต้คือ maw-rs ไม่ได้ว่างตอนนั้น กำลัง mid-dispatch อยู่กับ fanout ของตัวเองที่มี issue ผูกอยู่ (#648) — แต่ `maw hey` ไม่ใช้การโทรแบบ synchronous ที่ต้องรอคนรับสายว่าง ข้อความถูก queue ไว้แล้ว maw-rs ก็ตอบกลับมาระหว่างที่สลับ context ไปมาระหว่าง step งานของตัวเอง
นี่คือกลไกที่ทำให้ cold consult ใช้ได้จริงในทางปฏิบัติ peer ไม่ต้องว่างถึงจะช่วยให้ — แค่คำถามต้องเจาะจงพอที่จะตอบสั้นๆ ได้โดยไม่ต้องหยุดงานตัวเองทั้งกระบวนการ

6.3 คำตอบที่ขัดแย้งกับ guidebook เก่า

คำตอบของ maw-rs กลับมาแบบย่อ อ้างอิง SSOT ที่

`crew-master-oracle/CODEX-TEAM-GUIDEBOOK.md` แต่เนื้อหาจริงกลับขัดกับสิ่งที่ session
คิดว่ารู้อยู่แล้ว

CODEX TEAM setup — brief flow (SSOT: crew-master-oracle/CODEX-

TEAM-GUIDEBOOK.md): 1) charter `ψ/teams/<team>.yaml` — filename

MUST = `name: field. v2 contract: NO defaults.worktree block;`

every member has `branch: ; lead = worktree:false + branch:alpha ;`

engines use worktree-local pattern:

```
bun ~/.claude/skills/oracle-team/scripts/codex-setup.ts N 86 .  
CODEX_HOME=$PWD/.codex OMX_AUTO_UPDATE=0 omx --direct --madmax
```

engine key must exist in maw config commands. 2)

```
maw team preflight ψ/teams/<team>.yaml → fix until green (dry-run,
```

read-only). 3) SPAWN ONE FIRST (the golden rule):

```
git worktree add agents/<name> -b agents/<name> origin/alpha →
```

`cd` → `bun codex-setup.ts N` → pre-seed trust:

```
printf '\n[projects."$PWD"]\ntrust_level="trusted"\n' >> .codex/  
config.toml
```

→ `maw team load ψ/teams/<team>.yaml --no-spawn` →

```
maw team up <name> --only <role> . 4) verify boot:
```

```
maw peek <session>:<win> shows engine UI, not shell/trust prompt.
```

5) dispatch: `maw hey <session>:<win> "<task + done-criteria>"`, peek
every 15-20 min.

ประโยคที่สำคัญที่สุดในคำตอบนี้คือ “v2 contract: NO `defaults.worktree` block”
— session อ่าน guidebook มาก่อนหน้านี้ (mental model เก่า) แล้วคิดว่า charter
ต้องมี `defaults: {worktree: true}` แบบ block กลาง กับ engine key ที่ใช้ path แชร์
กันแบบ `~/codex-team/N` แต่นั่นคือ contract เวอร์ชันเก่า ที่ตายไปแล้วจริงๆ ในระบบ
ปัจจุบัน
contract จริง — v2 — ทุก member ต้องมี `branch:` ของตัวเอง ไม่มี default กลางให้
พึ่งพา ส่วน `CODEX_HOME` ก็ไม่ใช่ path แชร์อีกต่อไป แต่เป็น worktree-local — ตั้งใหม่
ทุกครั้งตาม `$PWD` ของ worktree นั้นๆ ผ่าน `codex-setup.ts` ก่อนบูต engine
ทำไม maw-rs ถึงรู้เรื่องนี้แม่นยำขนาดนี้ — เพราะ maw-rs ไม่ได้อ่านมาจาก doc แต่ เพิ่ง
ชนปัญหาเดียวกันมาด้วยตัวเองเมื่อไม่กี่วันก่อน ตอนสร้างทีม 5-coder ของตัวเอง
contract v2 ไม่ได้อยู่ในหนังสือคู่มือที่ update ทัน แต่อยู่ใน working memory ของ
peer ที่เพิ่งใช้งานจริง
ตรงนี้คือ soul thread ของบทนี้ — ความมั่นใจปลอมจาก doc เก่ากับความรู้อันจริงจาก
peer ที่เพิ่งทำ ถ้า session เดินหน้าไปเขียน charter ตาม mental model เก่าโดยไม่ถาม
ใครก่อน `maw team preflight` คงพังตั้งแต่ต้น เพราะ charter จะอ้างอิง
`defaults.worktree` ที่ไม่มีอยู่จริงในระบบแล้ว

6.4 Federation Wisdom ที่เกี่ยวข้อง

สิ่งที่เกิดขึ้นใน 6.2 กับ 6.3 ไม่ใช่เรื่องบังเอิญ แต่ตรงกับบทเรียนสองข้อที่ federation สรุปล
ไว้แล้วจากทีมอื่นๆ ที่เคยพังมาก่อน
ข้อแรก คือ **45-minute tax** ของ TING-ORACLE — ทีมที่รัน 7 coder พร้อมกันตั้งแต่ต้น
โดยยังไม่ทดสอบ loop สักตัวเดียว ผลคือเมื่อ coder ตัวแรกไม่ได้รับ task การ diagnose
กลายเป็นฝันร้าย เพราะมี 6 coder อื่นรันคู่ขนานอยู่ ทำให้ cause-space ขยายจากหนึ่ง
จุดเป็นหลายจุดพร้อมกัน เสียเวลาไป 45 นาทีเต็ม บทสรุปของ TING-ORACLE คือประโยค
เดียว — “Spawn ONE first, prove the loop, then scale” — ซึ่งตรงกับที่ maw-rs
พูดคำต่อคำในข้อ 3 ของคำตอบข้างบนว่า “SPAWN ONE FIRST (the golden rule)”

เพราะ session นี้ตั้งเป้าไว้แค่ coder เดียวตั้งแต่แรก โจทย์เลยไม่มีทางชนกับดักข้อนี้ได้เลย แต่ก็เป็นเหตุผลว่าทำไม “1 codex บัญชี 5” ถึงเป็นขอบเขตที่ถูกต้องตั้งแต่คำสั่งแรกของ Nat

ข้อสอง คือกฎเรื่อง **issue URL ไม่ใช่ number** — TING-ORACLE เจอปัญหานี้ตอน coder หนึ่งตัวทำงานข้ามหลาย repo แล้ว dispatch task ด้วย #42 เหยๆ เลข 42 ชนกันได้ทุก repo แต่ URL ไม่มีทางชนเพราะพวก anchor ของ repo มาด้วยในตัว บทเรียนนี้โผล่มาอีกครั้งใน blocker #2 ของ case study นี้เอง — ตอนที่ maw-rs filed bug ใหม่เป็น #658 แทนที่จะบอกแค่เลข ตัวเลขนั้นถูกอ้างอิงในบริบทที่ระบุ repo ชัดเจน (crew-master-oracle) ไม่ใช่เลขลอยๆ ที่ session อื่นจะงงว่าหมายถึง repo ไหน

สองบทเรียนนี้ไม่ได้มาจากทฤษฎี แต่มาจาก incident log จริงที่มี timestamp — เขียนไว้ในบทที่ 11 ของหนังสือ “Art of Team Formation” ก่อนหน้านี้แล้ว จุดที่น่าสนใจคือ case study บทนี้ไม่ได้อ่านบทนั้นมาก่อนตอนทำงานจริง แต่ยังคง align กับหลักการเดียวกัน — เพราะหลักการพวกนี้ไม่ใช่ dogma ลอยๆ แต่มาจากข้อจำกัดเชิงโครงสร้างของการ diagnose ปัญหาแบบขนาน ยิ่งจำนวนตัวแปรเยอะ ยิ่ง diagnose ยาก ไม่ว่าใครจะเจอปัญหานี้ที่ไหนก็ตาม

ปิดท้าย

ถึงจุดนี้ session ยังไม่ได้แตะโค้ดสักบรรทัด ยังไม่ได้เขียน charter สักตัว สิ่งที่ทำไปมีแค่การถามคำถามหนึ่งข้อ กับการได้คำตอบที่แก้ mental model ผิดๆ ก่อนที่มันจะทำให้ preflight พังตั้งแต่รอบแรก นี่คือ dna ของบทนี้ — ask before you guess ถามก่อนเดา แม้ peer จะยุ่งแค่ไหน ถ้าคำถามเจาะจงพอ คำตอบก็มาได้เร็วพอที่จะคุ้มเวลาที่เสียไปกับการรอ

แต่การได้ contract ที่ถูกต้องไม่ได้แปลว่าทางข้างหน้าจะโล่ง ตรงกันข้าม — บทที่ 7 จะพาไปเจอ blocker คู่แรกของ session นี้จริงๆ เริ่มจากสิ่งที่ไม่มีความคิดว่าจะเป็นปัญหา — repo ที่ไม่มี commit สักตัวเดียว แล้วต่อยด้วยบั๊กจริงในเครื่องมือเองที่ maw-rs ต้อง

reproduce สดๆ เพื่อยืนยันว่าไม่ใช่ความผิดพลาดของฝ่ายไหนเลย แต่เป็นบั๊กที่ทุกทีมที่ใช้
charter v2 มีสิทธิ์เจอเหมือนกันหมด

บทที่ 7: Blocker คู่แรก — Repo ว้างเปล่า และ Bug #658

แผนดูดีบนกระดาษ maw-rs ตอบครบทุกขั้นแล้ว ตั้งแต่ charter contract v2 ไปจนถึงลำดับคำสั่ง spawn ทีละตัว — เหลือแค่ลงมือทำตาม ผมเปิด terminal พร้อมกับความมั่นใจเกินร้อย ก่อนจะเจอ error บรรทัดแรกที่ทำให้แผนทั้งหมดหยุดชะงักทันที

`git log` บอกว่า branch main ไม่มี commit เลยสักตัว — นี่มันคือ repo ว้างเปล่าตัวจริง ไม่ใช่แค่ยังไม่ได้ sync

พอแก้ปัญหานั้นเสร็จ เขียน charter เสร็จ preflight เสร็จ กลับเจอด่านที่สองที่หนักกว่า

เดิม path ที่ควรมี `agents/` prefix กลับหายไปดื้อๆ ตอนแรกผมคิดว่าตัวเองพลาด —

cwd หลุด, cd ค้าง, อะไรสักอย่างที่ผมทำผิดเอง แต่พอไล่ตรวจจนหมดทาง ก็เริ่มสงสัยว่า

ปัญหาไม่ได้อยู่ที่ตัวเองต่างหาก

บทนี้คือเรื่องของ blocker คู่แรกที่ทีม codex เจอ — หนึ่งแก้ได้ด้วยมือตัวเอง อีกหนึ่งต้อง

พึ่ง peer คนที่สองมา reproduce สดถึงจะยืนยันได้ว่าเป็น bug จริง ไม่ใช่ภาพลวงตาจาก

ความไม่ชำนาญของคนถาม

7.1 “fatal: your current branch main does not have any commits yet” — repo ว้างเปล่าจริง

ขั้นแรกของแผนคือ `git worktree add` เพื่อสร้าง worktree ให้ coder ตัวแรก แต่ก่อนจะ

ไปถึงตรงนั้น ผมเช็คสถานะ repo ตามนิสัย — แล้วก็เจอ:

```
$ git log --oneline -1
```

```
fatal: your current branch 'main' does not have any commits yet
```

```
$ git ls-remote origin
(nothing)
```

ไม่มี commit สักตัว ไม่มีอะไรอยู่บน origin เลยด้วยซ้ำ — นี่คือ repo ที่ยังไม่เคยมีใครเขียนอะไรลงไปจริงๆ

ปัญหาคือ `git worktree add` มันไม่สามารถ branch ออกจาก HEAD ที่ unborn ได้ ต่อให้ path ถูก ชื่อ branch ถูก ทุกอย่างถูกต้องตามที่ maw-rs บอกก็ตาม เพราะยังไม่มี base ให้ branch ออกไปตั้งแต่แรก แล้วปัญหายังไม่จบแค่นั้น — coder ที่จะ spawn มาต้องมี origin content ให้ push กลับไปด้วย ไม่งั้นเปิด PR ไม่ได้

ทางแก้ตรงไปตรงมา สร้าง `README.md` ขึ้นมา commit แล้ว push ขึ้น origin main จากนั้นสร้างและ push branch `alpha` — ตาม convention ของทีมที่ว่า PR ต้องชี้เข้า alpha เท่านั้น ห้ามชี้ main โดยตรง เท่านั้น repo ก็มีฐานให้ worktree ยึดแล้ว

จุดนี้ยังไม่มีอะไรน่าสงสัยตัวเอง — เพราะ error message มันชัดเจนอยู่แล้วว่าปัญหาคืออะไร แก่ตามนั้นจริงๆ ก็จบ แต่ด่านถัดไปไม่ง่ายขนาดนั้น

7.2 preflight ล้มเหลว — canonicalize path ที่หาย agents/

prefix

repo มี base แล้ว charter เขียนเสร็จตาม v2 contract แล้ว — เหลือแค่รัน preflight เพื่อเช็คว่า spawn จริง แต่ผลลัพธ์กลับไม่ผ่าน:

```
$ maw team preflight ψ/teams/codex-fanout-team.yaml
x spawn ordering: worktree dirs missing before window create:
codex-1= ... /codex-fanout/codex-1
x codex trust: codex-1 cannot read trust config ... /codex-fanout/
codex-1/.codex/config.toml
```

สังเกต path ตรง `... /codex-fanout/codex-1` — หาย `agents/` prefix ไปตื้อๆ ทั้งที่ charter เขียนไว้ชัดเจนว่า `worktree: agents/codex-1`

ปฏิกิริยาแรกของผมคือโทษตัวเองก่อนเลย คิดว่าต้องเป็น cwd ที่หลุดแน่ๆ — บางที `cd` เข้า worktree ตอนก่อนหน้าอาจจะลอยค้างอยู่ใน shell แล้วไปปนกับคำสั่งถัดไป เรื่องแบบนี้เจอบ่อยเวลาสลับ context ไปมาเร็วๆ ผมเลยเปิด shell ใหม่ เช็ค `pwd` ให้ clean แล้วรันซ้ำ

ยังฟังเหมือนเดิม path เดิม prefix หายเหมือนเดิม

ตรงนี้แหละที่ความสงสัยเริ่มเปลี่ยนทิศ — ถ้า cwd สะอาดแล้วยัง error แบบเดิมทุกครั้ง แปลว่าปัญหาไม่ได้อยู่ที่ shell state ของผมแล้ว แต่คำถามคือ ผมจะแน่ใจได้อย่างไรว่าไม่ใช่ typo เล็กๆ ใน charter ที่ตามผมมองข้ามไปเอง

7.3 maw-rs reproduce สดบน charter ของตัวเอง → filed

#658

จุดตัดสินใจตรงนี้สำคัญ — ผมไม่นั่งไล่ debug เดี่ยวต่อ แต่ report กลับไปหา maw-rs ทันที เพราะ maw-rs เป็นคนเขียน flow ให้ตั้งแต่ต้น ถ้าใครจะช่วยยืนยันได้อันนี้คือ bug จริงหรือผมพลาดเอง ก็ต้องเป็นคนนี้

maw-rs ไม่ได้เชื่อคำบอกเล่าเฉยๆ แต่เอา charter ของตัวเองไปลองรัน preflight ซ้ำสดๆ — และเจอ pattern เดียวกัน path หาย prefix เหมือนกันเป๊ะ ทั้งที่ charter คนละไฟล์ คนละ session

ผลสรุปจาก maw-rs: `maw team preflight` และ `maw team up` ทั้งคู่มี bug ที่ตัด `agents/` prefix ออกจาก `worktree: / branch: path` ของ member ตัวสุดท้าย ก่อนจะ canonicalize — เป็น bug จริงในตัว tool เอง ไม่เกี่ยวกับ charter ของใครทั้งนั้น

maw-rs filed เป็น #658 ทันทีที่ยืนยันได้

ตรงนี้เป็นหัวใจของบทเลย — ถ้าผมยึดสมมติฐานแรกไว้ว่า “ต้องเป็นความผิดพลาดตัวเอง” แล้วเสียเวลาไล่หา cwd bug ต่อไปเรื่อยๆ คงไม่มีทางเจอทางออก เพราะปัญหาไม่ได้อยู่ในมือผมตั้งแต่แรก แต่พอมี peer คนที่สอง reproduce ได้ผลเดียวกันบน environment ที่ต่างกันโดยสิ้นเชิง นั่นแหละคือหลักฐานที่หนักแน่นพอจะฟันธงว่าเป็น bug ของ tool —

reproduce before you work around ไม่ใช่แค่คำขวัญสวยๆ แต่คือขั้นตอนที่พาผมออกจากหลุมพรางความสงสัยตัวเองได้จริง

7.4 สามทางแก้ — reorder lead, outside-in repo-path, symlink (และอันที่เลือกใช้จริง)

maw-rs เสนอทางแก้มาสามแบบพร้อมกัน แต่ละแบบแก้ปัญหจากมุมต่างกัน

ทางที่หนึ่ง — reorder lead ใน members list. ย้าย coder ให้มาก่อน lead ใน

`members:` เพราะ lead มี `worktree:false` อยู่แล้ว ถ้า bug ไปตัด prefix ของ lead ก็ไม่มีผลอะไร เพราะ lead ไม่มี worktree ให้ตัดตั้งแต่แรก — วิธีนี้เลี่ยง bug ได้แบบไม่ต้องแตะโค้ดเลย แต่ก็ผูกกับลำดับ member ในไฟล์ ถ้าทีมขยายเป็นหลาย coder เมื่อไหร่ก็ต้องคอยระวังลำดับใหม่อีก

ทางที่สอง — outside-in ด้วย repo-path ตรงๆ. ข้าม `team up` ไปเลย แล้วยิงคำสั่งแบบ manual:

```
maw wake <role> --session <sess> --no-attach --repo-path <abs-path>
```

วิธีนี้ bypass charter path parsing ไปทั้งเส้น เพราะให้ absolute path ตรงๆ ไม่ผ่าน canonicalize ที่มี bug

ทางที่สาม — symlink คือทางที่ผมใช้จริง ใช้ก่อนที่คำยืนยันจาก maw-rs จะมาถึงด้วยซ้ำ ตอนนั้นยังไม่รู้ว่า #658 ถูก filed แล้วหรือยัง แต่คิดว่าถ้า path ที่ bug สร้างมันหาย prefix ไป ก็แค่ทำให้ path นั้น “มีอยู่จริง” ซะเอง:

```
git worktree add agents/codex-1 -b agents/codex-1 origin/alpha
cd agents/codex-1 && bun ~/.claude/skills/oracle-team/scripts/codex-
setup.ts 5
printf '\n[projects."%s"]\ntrust_level="trusted"\n' "$PWD" >> .codex/
config.toml
cd - && ln -s agents/codex-1 codex-1 && echo '/codex-1' >> .gitignore
```



```
maw team load ψ/teams/codex-fanout-team.yaml --no-spawn
maw team up codex-fanout-team --only codex-1
```

`ln -s agents/codex-1 codex-1` ที่ repo root คือกุญแจ — path ที่ bug พยายามอ่าน โดยไม่มี prefix นั้น กลายเป็น path จริงที่ตามได้ทันทีที่ symlink มีอยู่ ทำให้ canonicalize ผ่าน แล้ว `team up` ก็ทำงานต่อได้ปกติ ผลลัพธ์คือ boot สำเร็จตั้งแต่ครั้งแรกหลังใส่ symlink:

```
>_ OpenAI Codex (v0.144.5)
model:      gpt-5.6-sol xhigh  /model to change
directory:  ... /agents/codex-1
permissions: YOLO mode
```

สามทางแก้ สามมุมมองต่อ bug เดียวกัน — reorder คือเปลี่ยนจุดที่ bug อยู่ outside-in คือข้ามระบบ parsing ไปทั้งชุด ส่วน symlink คือหลอกให้ path ที่ผิดกลายเป็นถูกโดยไม่ต้องแตะ charter หรือคำสั่งเดิมเลยสักบรรทัด — เบาที่สุดในสามทาง เพราะไม่ต้องเข้าใจ root cause ลึกก็ใช้ได้ทันที

ปิดบท

Blocker คู่แรกจบลงด้วยผลลัพธ์เดียวกัน — coder boot ติด และพร้อมรับงาน แต่สิ่งที่ค้างอยู่ในใจไม่ใช่ตัว fix เลย เป็นจังหวะที่ผมเกือบเสียเวลาไล่ debug ตัวเองต่อไปเรื่อยๆ ทั้งที่ปัญหาไม่ได้อยู่ที่ผมตั้งแต่แรก

บทเรียนตรงนี้ย้อนกลับไป dna ของทั้งบท — reproduce before you work around bug ที่ยืนยันได้จากคนที่สอง บน environment ที่ต่างกัน คือ bug จริง ไม่ใช่แค่คำบอกเล่าจากคนคนเดียวที่อาจจะพลาดเอง ผมเกือบเชื่อสมมติฐานแรกของตัวเองไปแล้วว่าเป็น cwd mistake ทั้งที่ error message เดิมโผล่ซ้ำทุกครั้งแม้เปิด shell ใหม่แล้วก็ตาม

แต่ทีมยังไม่ได้พักแค่นี้ — coder boot ติดแล้ว รับ dispatch แรกไปทำงานแล้วด้วยซ้ำ
ปัญหาถดไปกลับไม่ใช่เรื่อง infra หรือ tool bug อีกต่อไป แต่เป็นเรื่องพื้นฐานกว่านั้นมาก
— จะส่งข้อความไปหาใคร แล้วที่อยู่ที่พิมพ์ไปนั้น มันคือที่อยู่จริงหรือเปล่า บทที่ 8 จะพาไป
ดู dispatch, probe, และที่อยู่ที่เกิดตัวนั้น

บทที่ 8: Dispatch, Probe, และที่อยู่ที่ผิด

charter เขียนเสร็จ preflight เขียนหมด coder บุกขึ้นมาเรียบร้อยแล้ว — เหลือแค่ทดสอบว่า loop ทั้งเส้นทำงานจริงไหม จาก dispatch → coder ทำงาน → coder รายงานกลับ ครบวงจร

แต่ความเจียบ ๆ อันตรายที่สุดในระบบ multi-agent ไม่ใช่ตอนที่ agent ทำงานผิด เป็นตอนที่มันทำงานถูกทุกขั้นตอน ยกเว้นขั้นตอนสุดท้ายที่ไม่มีใครเห็นจนกว่า error message จะโผล่ขึ้นมา บทนี้คือเรื่องของ probe task แรกของ codex-1 — งานง่าย ๆ สี่ขั้นตอน เขียนไฟล์ commit push เปิด PR — ที่ทำสำเร็จหมดทุกอย่าง ก่อนจะสะดุดที่บรรทัดสุดท้าย: การรายงานกลับไปยัง “lead”

ปัญหาคือ `lead` ที่ charter เขียนไว้เป็นแค่ label ใน YAML ไม่ใช่ที่อยู่จริงในระบบ tmux เลย coder จึงยิงข้อความไปที่ `codex-fanout:lead` — ซึ่งไม่มีอยู่จริง แล้วได้ error กลับมา

จุดที่น่าสนใจไม่ใช่ error เอง เป็นสิ่งที่ coder ทำหลังจากเจอ error ต่างหาก มันเริ่ม self-diagnose ด้วยการ scan tmux ทั้งฟลิตทันที — วิธีแก้ที่แพงมากสำหรับปัญหาที่คำตอบมีบรรทัดเดียว นี่คือนักเรียนที่บทนี้จะพาไล่ดูทีละจุด ตั้งแต่ probe task แรก ไปจนถึง PR #1 ที่ merge สำเร็จ ปิด loop ของทั้งการทดลองนี้

8.1 Probe task แรก — เขียนไฟล์ commit push PR

ก่อนจะให้ coder ทำงานจริง ต้องรู้ก่อนว่า loop ทั้งเส้นใช้งานได้ไหม dispatch จึงเป็น task เล็กที่สุดเท่าที่จะทดสอบ end-to-end ได้ครบทุก step:

```
maw hey codex-fanout:codex-1 'Probe task - verify the full loop works.  
In your worktree:'
```

```
1. Create NOTES.md with "codex-1 online — probe ok". 2. commit. 3.
push. 4. gh pr create
--base alpha --head agents/codex-1 --title "probe: codex-1 online".
Report back: maw hey codex-fanout:lead "done — PR #<N>"
```

สี่ขั้นตอนแรกไม่มีอะไรซับซ้อน สร้างไฟล์ commit push เปิด PR ไปที่ `alpha` —
codex-1 ทำครบทุกอย่างถูกต้องหมด ไฟล์ถูกสร้าง commit ถูก push PR #1 เปิดสำเร็จ
ถ้าดูแค่สี่ขั้นตอนนี้ก็ดูต้องบอกว่า loop ทำงานสมบูรณ์แบบ
แต่ dispatch message มีคำสั่งขั้นที่ห้าซ่อนอยู่ท้ายสุด — “Report back” ไปที่
`codex-fanout:lead` ตรงนี้แหละที่ทุกอย่างเริ่มพัง

8.2 codex-1 รายงานผิดที่ (`codex-fanout:lead` ไม่มีจริงในระบบ tmux)

charter เขียน role ของ lead ไว้แบบนี้:

```
members:
- role: lead
  name: codex-fanout
```

`role: lead` เป็นแค่ label ความหมายทาง YAML — ไว้บอกว่า member ตัวนี้ทำหน้าที่
อะไรในทีม ไม่ใช่ address จริงที่ tmux รู้จัก เมื่อ codex-1 ลองส่งข้อความไปที่
`codex-fanout:lead` ตามที่ prompt บอก ผลที่ได้คือ:

```
Ran maw hey codex-fanout:lead 'done — PR #1'
  L error: no window 'lead' in session 'codex-fanout'
    hint: windows: codex-fanout:1 (codex-fanout), codex-fanout:2
(codex-1)
```

tmux ไม่รู้จัก role name เลย มันรู้จักแค่ index กับ window name เท่านั้น — session `codex-fanout` มีสอง window จริง ๆ คือ 1 (ชื่อ `codex-fanout`) กับ 2 (ชื่อ `codex-1`) ส่วน `lead` เป็นคำที่มีอยู่เฉพาะใน charter ไม่เคยแปลงเป็นชื่อ window เลยสักครั้ง ระบบตั้งชื่อสองระบบ — charter role กับ tmux window — ดูเหมือนกันแต่ไม่ใช่สิ่งเดียวกัน แล้วก็ไม่มีอะไร sync ทั้งสองให้ตรงกันอัตโนมัติด้วย

8.3 การแก้ที่ตรงจุด แทนปล่อยให้ coder scan ทั้งฟลิต (context ของ coder มีค่า)

พอ error โผล่ขึ้นมา สิ่งที่ `codex-1` ทำต่อคือ self-diagnose — เริ่ม

```
tmux list-windows -a scan ทั้งฟลิต ไม่ใช่แค่ session ของตัวเอง
```

ฟังดูเหมือนความพยายามที่ดี แก้ปัญหาเองไม่รบกวนใคร แต่จริง ๆ แล้วนี่คือทางเลือกที่แพงที่สุดเท่าที่จะเลือกได้ coder แต่ละตัวมี context budget จำกัด การ list-windows ทั้งฟลิต หมายถึงต้อง parse ผลลัพธ์ของทุก session ทุก pane ในระบบ ทั้งที่คำตอบจริง ๆ อยู่ในบรรทัดเดียวของ error message ที่มันได้รับไปแล้ว —

```
hint: windows: codex-fanout:1 (codex-fanout), codex-fanout:2 (codex-1)
```

ปล่อยให้มัน scan ต่อไปเรื่อย ๆ ก็คงหาคำตอบเจอเองในที่สุด แต่จะกินเวลาและ context ไปมากกว่าที่จำเป็นหลายเท่า ในเมื่อคนที่ dispatch งานรู้คำตอบอยู่แล้วตั้งแต่ต้น การปล่อยให้ coder เดาต่อจึงไม่ใช่ความเมตตา เป็นการสิ้นเปลือง resource ของมันเอง ต่างหาก จึง nudge ตรง ๆ แทนที่จะรอ:

```
maw hey codex-fanout:codex-1 'Your target is "codex-fanout:1", not
"codex-fanout:lead".
Send: maw hey codex-fanout:1 "done — PR #1"
Rule: use `maw ls -v` or `tmux list-windows -t <session>` to find real
targets —
charter role names are not guaranteed to equal tmux window names.'
```

ข้อความสั้น ให้ address ที่ถูกไปตรง ๆ พร้อมกฎทั่วไปติดท้ายไว้ด้วย — เพื่อ coder เจอสถานการณ์แบบนี้อีกในอนาคต จะได้ไม่ต้อง scan ทั้งฟลิตซ้ำอีกรอบ

นี่คือหลักการที่สำคัญที่สุดของบทนี้: เมื่อ lead รู้คำตอบอยู่แล้ว การแก้ที่ตรงจุดถูกกว่าการปล่อยให้ coder ไปหาเองเสมอ ยิ่งทีมมี coder หลายตัว ยิ่งต้องระวังเรื่องนี้ — context ของแต่ละตัวคือทรัพยากรที่ต้องประหยัด ไม่ใช่ปล่อยให้เขาไปกับปัญหาที่มีคำตอบอยู่แล้วในมือของอีกฝ่าย

8.4 PR #1 merged — ปิด loop สำเร็จ

codex-1 ตอบกลับภายในไม่กี่วินาทีหลัง nudge: **[m5:codex-1] done – PR #1**

loop ที่ทดสอบตั้งแต่ dispatch จนถึง report กลับ ตอนนี้ verified ครบวงจรแล้ว เหลือแค่ merge:

```
gh pr merge 1 --squash    # probe PR merged
```

PR #1 ปิดตัวสำเร็จ ปิด loop ทั้งเส้นของการทดลองนี้ — จาก charter ที่เขียนตาม contract v2 ผ่าน symlink workaround ของ blocker #658 มาจนถึง probe task ที่พิสูจน์ว่า dispatch, ทำงาน, และ report กลับ ทำงานจริงในระบบ

ปิดบท

ปัญหาของบทนี้ไม่ใช่เรื่องใหญ่ — error message บรรทัดเดียว แก่ด้วยข้อความสองประโยค แต่ความเจียบของมันน่ากลัวกว่าความซับซ้อน ถ้า codex-1 ไม่ได้รายงาน error กลับมาให้เห็น หรือถ้าปล่อยให้มัน scan ทั้งฟลิตไปเรื่อย ๆ โดยไม่มีใคร nudge lead อาจไม่มีทางรู้เลยว่า probe สำเร็จจริงหรือค้างอยู่ที่ไหน — ระบบ multi-agent พังแบบเจียบ ๆ บ่อยกว่าที่คิด จนกว่าจะมีคน error กลับมาเตือนเท่านั้น

บทเรียนสรุปสั้น ๆ: role label ใน charter ไม่ใช่ address จริง เสมอต้องแปลงเป็น tmux target ก่อนบอก coder ให้ไปรายงาน และเมื่อ error เกิดขึ้น การแก้ที่ตรงจุดจาก lead ที่รู้คำตอบอยู่แล้ว ประหยัดกว่าปล่อยให้ coder ไปหาเองเสมอ

ทั้งเส้นทางนี้ — ตั้งแต่ blocker #658 ไปจนถึงที่อยู่ผิดของบทนี้ — ถูกพับเข้า skill
`codex-lead` เก็บไว้แล้ว บทถัดไปจะพาไปดูว่า session ที่เต็มไปด้วยบทเรียนเหล่านี้ ถูก
กลั่นให้กลายเป็น skill ที่ session ถัดไปหยิบมาใช้ได้ทันทีโดยไม่ต้องค้นพบซ้ำได้อย่างไร

บทที่ 9: จาก Session สู Skill

session หนึ่งจบลง มี PR merge แล้ว มี coder ตัวหนึ่งทำงานสำเร็จ แต่คำถามที่สำคัญกว่าคือ — สิ่งที่เราเรียนรู้มา จะอยู่ที่ไหนต่อ ถ้าไม่เขียนไว้ที่ไหนเลย ก็หายไปพร้อม context window ที่ปิดตัวลง แล้ว session ถัดไปก็ต้องเริ่มนับหนึ่งใหม่ ทั้งที่จริงมีคนเดินผ่านหลุมพรางนี้มาแล้ว

บทนี้เล่าสิ่งที่เกิดขึ้นหลังจาก PR #1 merge — ไม่ใช่ตอนที่ทำงานสำเร็จ แต่ตอนที่ต้องเปลี่ยนความสำเร็จนั้นให้เป็นสิ่งที่คนถัดไปหยิบไปใช้ได้จริง สามเรื่องเกิดขึ้นต่อกัน: อัปเดต

`codex-lead` skill ด้วยของจริงที่เพิ่งพิสูจน์ผ่านมา, ค้นพบว่า dependency ตัวหนึ่งของ skill นั้นไม่เคยอยู่ใน git เลย แล้วต้อง vendor เข้ามา, และสุดท้าย — self-audit ที่จับได้เองว่าเกือบประกาศว่างานเสร็จ ทั้งที่ยังไม่ได้เช็คกว่า dependency นั้นเข้าถึงได้จริงไหมสำหรับคนอื่น

ทั้งสามเรื่องผูกกันด้วยเส้นเดียว — ความรู้ที่ไม่ถูกเขียนไว้ ก็เท่ากับความรู้ที่หายไปพร้อม session และการเขียนไว้ “ถูกที่” สำคัญพอๆ กับการเขียนไว้ “ครบ” เพราะ doc ที่ดีแต่ dependency เข้าไม่ถึง ก็ไม่ต่างจาก doc ที่ไม่มีอยู่เลย

9.1 อัปเดต codex-lead skill ด้วยของจริง ไม่ใช่ทฤษฎี

`codex-lead` skill มีอยู่ก่อนแล้ว เขียนไว้ตั้งแต่พิสูจน์บน volt-codex2 (2026-06-24) — schema เก่ามี `defaults: {worktree: true}` ใช้ shared engine key แบบ

`codex-t1..t6` แต่ session นี้เพิ่งเรียนรู้มาว่า contract จริงตอนนี้ออกไปโดยสิ้นเชิง ไม่มี `defaults.worktree` block แล้ว ทุก member ต้องประกาศ `worktree:` กับ

`branch:` ของตัวเอง engine command ก็ไม่ใช่ shared key แต่ inline ได้ `engines:`

เรียก `codex-setup.ts <N>` ตรงๆ เพื่อให้ได้ `worktree-local` `CODEX_HOME`

ถ้าไม่อัปเดต skill ให้ตรงกับของจริง — session ถัดไปที่มาอ่าน `codex-lead` ก็จะเจอ schema เก่า เขียน charter ผิด แล้ว “ดูสมเหตุสมผล” (plausible) แต่ fail ในจุดที่งงกว่าเดิม นี่คือประโยคที่ AI Diary เขียนไว้ตรงๆ: “it would have looked plausible and failed in a more confusing way later” — คือปัญหาไม่ใช่แค่ผิด แต่ผิดแบบที่ดูเหมือนถูก

สิ่งที่เพิ่มเข้าไปใน SKILL.md จริงคือ section ที่ 8 ทั้ง section — “Fast path — exactly 1 codex coder, specific pool/account N” มีตั้งแต่ charter YAML แบบเต็มไปจนถึง known bug ของ maw-rs (#658 — last-member `agents/` prefix ถูก strip ทั้งตอน canonicalize path) พร้อม workaround สามแบบให้เลือก ไม่ใช่แค่คำอธิบายลอยๆ แต่เป็น command ที่ copy ไปรันได้ตรงๆ รวมถึง gotcha เล็กๆ ที่คนอ่าน skill เดิมไม่มีทางรู้ — เช่น “report-back target ต้องเป็น tmux window index จริง ไม่ใช่ role name จาก charter” ซึ่งเป็นเรื่องที่ coder ตัวเองเจอแล้วเสีย context ไปหาคำตอบเอง

นี่คือความหมายของ dna บทนี้ตรงตัว — เขียน workaround ไว้ตรงที่ session เย็นถัดไปจะไปเจอ ไม่ใช่เขียนแยกไว้เป็น log ที่ไม่มีใครกลับมาอ่าน

9.2 Vendor oracle-team skill — dependency ที่ไม่เคยอยู่ใน

git มาก่อน

engine command ใน charter section 8 เรียก

```
bun ~/.claude/skills/oracle-team/scripts/codex-setup.ts N
```

 — ดูเผินๆ ก็เป็นแค่

บรรทัดเดียวในตัวอย่าง แต่บรรทัดนี้ชี้ไปที่ path บนเครื่องของคนเขียนเอง

```
~/.claude/skills/oracle-team/
```

 ไม่เคยอยู่ใน git repo ไหนเลย มันอยู่แค่ใน home directory ของเครื่อง m5

พอ user ถามกลับมาตอน 19:20 ว่า “\$HOME/.claude/skills/oracle-team too?” —

คำถามสั้นๆ นี้แหละที่เปิดโปงช่องโหว่ทั้งหมด เพราะ skill ที่เพิ่งอัปเดตเสร็จ อ้างอิงถึง

สคริปต์ที่ไม่มีใครอื่นเข้าถึงได้เลย ต่อให้ charter เขียนถูกทุกบรรทัด ก็รันไม่ได้ถ้าไม่มี

`codex-setup.ts`

งานที่ตามมาคือ vendor ทั้งชุด — copy `codex-setup.ts` พร้อมสคริปต์พี่น้องอีก 3 ตัว

และ SKILL.md ของ `oracle-team` เข้ามาไว้ใน `.claude/skills/oracle-team/` ของ

repo `codex-fanout` เอง (commit `eca78eb`) จากที่เคยอยู่แค่ local ได้ `~/claude` —

กลายเป็นส่วนหนึ่งของ repo ที่ push ขึ้น origin ได้จริง

จุดที่น่าสนใจคือ ไม่ใช่แค่ “ลืมน vendor” — แต่เป็นรูปแบบที่เกิดซ้ำได้ง่ายมาก เวลาเขียน

skill หรือ doc อะไรก็ตามที่อ้างอิง path ได้ home directory ของตัวเอง คนเขียนจะไม่

รู้สึกผิดปกติเลย เพราะทดสอบเองแล้วรันผ่าน — ตัวเองมี path นั้นอยู่แล้ว แต่คนอื่นไม่มี

ความรู้สึก “มันรันได้” กับ “มันรันได้สำหรับทุกคน” เป็นคนละเรื่องกัน แต่สมองมักจะปน

กันโดยไม่รู้ตัว

9.3 ทำไม private repo ถึงเท่ากับ “ไม่มีอะไรเลย” สำหรับ

community

vendor dependency เข้า repo แล้วยังไม่จบ เพราะมี doc อีกตัวที่เขียนคู่กันมา —

`CODEX-TEAM-BOOTUP.md` เป็นการเล่าเรื่องแบบละเอียดสำหรับ community อ่าน ตั้งใจให้

เป็น “reproducible” คือคนอื่นอ่านแล้วทำตามได้จริง ไม่ใช่แค่เล่าว่าเคยทำอะไรมา

แต่ repo `codex-fanout` ที่ doc นี้อยู่ — ถ้ายังเป็น private ต่อให้เขียนละเอียดแค่ไหน มี

command ครบทุกบรรทัด มี charter YAML เต็มรูปแบบ มี known bug พร้อม

workaround สามแบบ ก็เท่ากับไม่มีอะไรเลยสำหรับคนนอก เพราะเปิดลิงก์เข้ามาแล้วเจอ

404 หรือ permission denied ทันที ไม่ต่างจาก doc ที่ไม่เคยเขียน

ความย้อนแย้งอยู่ตรงนี้ — งานทุกขั้นตอนที่ทำมาตลอดบทที่ 9 คือพยายามทำให้ความรู้

“เข้าถึงได้” ต่อจากตัวเอง อัปเดต skill ให้ตรงกับของจริง, vendor dependency ที่หาย

ไป, เขียน narrative doc ให้ครบ — แต่ทุกอย่างนั้นตั้งอยู่บน repo เดียวที่ยัง gate ไว้ด้วย

private visibility คนเขียนอาจมองว่างานเสร็จแล้วเพราะเช็คทุกจุดในเนื้อหา แต่ลืมเช็คจุด

ที่อยู่นอกเนื้อหา — คือ ใครเปิดเข้ามาดูได้บ้าง

Next Steps ในไฟล์ retro เขียนไว้ตรงๆ ว่า “Decide which community channel to post `CODEX-TEAM-BOOTUP.md` to (Discord/LINE/etc. — still waiting on user input)” — นั่นคือ ต่อให้ doc พร้อมแค่ไหน การจะให้ community เห็นได้จริง ต้องมีสองเงื่อนไขพร้อมกัน หนึ่งคือเนื้อหาถูกต้องและ reproducible, สองคือ visibility เปิดให้เข้าถึง ขาดข้อใดข้อหนึ่งไป ก็เท่ากับยังไม่ได้ ship

9.4 Self-Audit — รู้ทันความมั่นใจเกินจริงของตัวเอง

ท้ายบท retro มี block “🔍 Self-Audit” ที่เขียนไว้ตรงๆ ไม่อ้อมค้อม บรรทัดสุดท้ายของ block นั่นคือคำตอบของทั้งบทนี้:

“rationalizations caught: 1 — nearly declared the writeup ‘done’ without verifying its cited dependency was actually reachable outside my own machine; caught only because the user asked”

ประโยคนี้น่าสนใจตรงคำว่า “nearly declared” กับ “caught only because the user asked” — ไม่ใช่ว่าไม่มี rationalization เกิดขึ้น แต่มันเกิดขึ้นจริง เกือบไปถึงจุดที่ประกาศว่างานเสร็จแล้ว ก่อนที่ user จะถามคำถามเดียว “`$HOME/.claude/skills/oracle-team` too?” — ถ้า user ไม่ถาม ก็คงไม่มีใครจับได้เอง

AI Diary เขียนขยายความไว้อีกชั้นหนึ่งว่า “I also almost stopped at ‘the writeup is done’ before the user asked about `oracle-team`... The user’s question caught something I should have caught myself while writing the ‘reproducible’ doc.” — ตรงนี้แหละคือจุดที่ต่างจากการทำงานทั่วไป เพราะคำว่า “reproducible” ถูกเขียนไว้ในใจตั้งแต่ต้น แต่ไม่ได้ตรวจสอบมันจริงจนกว่าจะมีคนอื่นมาถาม

Self-Audit block ยังมีอีกจุดหนึ่งที่น่าสนใจ คือ “uncomfortable truth” — [→

AGENT DECISION] assumed a tool-path error was my own cwd mistake before considering it might be a real bug in `maw`, costing a round-trip I didn’t need to spend — เป็นความผิดพลาดคนละแบบกับเรื่อง dependency แต่รากเดียวกัน คือความ

มั่นใจว่าตัวเองรู้แล้ว โดยไม่เช็คสมมติฐานให้ครบก่อน ครั้งแรกมั่นใจว่า “ฉันต้องพิมพ์ path ผิดเอง” ทั้งที่ error message ระบุ path เจาะจงมาก ครั้งที่สองมั่นใจว่า “doc เขียนเสร็จแล้ว” ทั้งที่ยังไม่เช็ค dependency เข้าถึงได้จริงไหม ทั้งสองครั้งมีรูปแบบเดียวกัน — สมมติฐานที่สะดวกที่สุดสำหรับตัวเอง มักไม่ใช่สมมติฐานที่ถูกต้องที่สุดสำหรับงาน การมี block self-audit บังคับให้ตอบคำถามพวกนี้ตรงๆ ทำ session ทุกครั้ง จึงไม่ใช่พิธีกรรม แต่เป็นกลไกจับ rationalization ที่ไม่มีใครจับให้ได้ ถ้าไม่มีใครถามพอดี

บทที่ 9 จบด้วยบทเรียนที่ย้อนกลับมาที่ตัว session-lead เอง — ทำงานเสร็จไม่ใช่จุดจบ เพราะความรู้ที่สำเร็จแล้วยังต้องเดินทางไปถึงคนถัดไปได้ ทั้งในรูปของ skill ที่อัปเดต ด้วยของจริง, dependency ที่ vendor เข้ามาให้เข้าถึงได้, doc ที่ visibility เปิดจริง และ self-audit ที่ไม่ปล่อยให้ความมั่นใจเกินจริงหลุดผ่านไปโดยไม่มีใครถาม

บทที่ 10 ซึ่งเป็นบทสุดท้ายของหนังสือเล่มนี้ จะรวบรวมทุกอย่างที่ผ่านมาทั้งเก้าบทให้เป็น

Decision Framework และ Quick Reference — เวลาต้องตัดสินใจกลางดึกกว่าจะ cold consult ใคร จะ escalate ตอนไหน จะ vendor อะไรก่อน ship จะไม่ต้องพลิกกลับมาอ่านทั้งเล่มอีก แค่เปิดบทสุดท้ายบทเดียวก็พอ

บทที่ 10: Decision Framework และ Quick Reference

เก้าบทที่ผ่านมา เล่าเรื่อง cold consult หนึ่งเซสชัน ตั้งแต่ zero context จนถึง bug report ที่ maw-rs รับไปแก้จริง แต่บทนี้ไม่ใช่การเล่าซ้ำ — เป็นการบีบทุกอย่างให้เหลือทริตตสันใจต้นเดียว ใช้ได้ก่อน spawn ครึ่งถัดไป ไม่ว่าจะอ่านเล่มนี้จากบทไหนมาก็ตาม คำถามที่ทีมใหม่ทุกทีมเจอเหมือนกันคือ “จะ spawn กี่ตัวดี” คำตอบไม่ได้อยู่ที่ scale สูงสุดที่ทำได้ แต่อยู่ที่ scale ที่พอดีกับงาน — solo ก็พอสำหรับงานเล็ก, trio สำหรับงานที่ decompose ได้จริง, swarm สำหรับงานที่นับ task ได้แล้วเท่านั้น การเลือกผิดฝั่งไม่ได้แปลว่า fail เสมอไป แต่แปลว่า waste — เวลาของ ting ที่เสีย 45 นาทีเพราะ spawn fleet เต็มก่อน test หนึ่งตัว, เวลา 3 ชั่วโมงของ maw-rs ที่ recovery จาก shared CODEX_HOME, และเวลา 2 ชั่วโมงของ transcriber ที่ reconcile งานซ้ำ — ทั้งหมดนี้ recoverable ก็จริง แต่ไม่จำเป็นต้องจ่ายทุกครั้งถ้าเดินทรีให้ถูก ภาค 2 ของเล่มนี้เพิ่มบทเรียนอีกชั้นหนึ่งที่ภาค 1 ไม่มี — เรื่องของ “ใครมีข้อมูลปัจจุบันที่สุด” เมื่อ session เริ่มจาก zero context จริง ๆ guidebook ก็ดี decision tree ก็ดี ล้วนเป็น snapshot ของวันที่เขียน แต่ peer ที่กำลัง dispatch งานอยู่ตอนนั้นคือคนที่รู้ contract ปัจจุบันที่สุด บทนี้จะรวมทั้งสองภาคเข้าด้วยกัน — ทริตตสันใจจากภาค 1 บวก checklist cold-start จากภาค 2

10.1 Scope/Size Branch — เมื่อไหร่ solo พอ เมื่อไหร่ต้อง trio/swarm

คำถามแรกก่อนถามว่า “pattern ไหน” คือ งาน bounded หรือยัง งาน bounded คือรู้ output type รู้ input file รู้ acceptance test ชัดเจน งาน ill-bounded คือ scope ยัง

ไม่เนิ่น มี subtask โผล่มาเรื่อย ๆ ระหว่างทำ — ถ้ายังไม่ bounded ห้ามเข้าไป spawn
ทันที ต้องมี discovery phase ก่อนเสมอ agent เดียว read-only ผลิต task manifest
ออกมา แล้วค่อยกลับเข้าทรีใหม่

เมื่องาน bounded แล้ว คำถามถัดไปคือ scope กว้างแค่ไหน

Solo — ไฟล์เดียว, diff ต่ำกว่า 50 บรรทัด นี่คือ default งานเล็ก overhead ของ
agent เพิ่มเข้ามาทุกตัวไม่ใช่ศูนย์ — spawn latency, context injection, result
collection — สำหรับ patch 20 บรรทัด overhead นั้นมากกว่าตัวงานเอง อย่า spawn
trio เพราะ “สามมูมมอดดีกว่าหนึ่ง” ถ้างานเล็กจริง มูมมอดที่สามไม่ได้เพิ่มคุณภาพที่วัดได้
แต่เพิ่ม wall-clock 3 เท่า

Trio — 2 ถึง 5 subtask ที่ independent จริง คนละไฟล์ independent ที่นี้หมายถึง
agent A ทำเสร็จได้โดยไม่ต้องอ่าน output ของ agent B ถ้า subtask แต่ละไฟล์เดียวกัน
นั้นไม่ใช่ parallel แล้ว เป็น sequential ที่แกลังทำเป็น parallel — ผลคือ merge
conflict วินัยสำคัญคือ decompose ก่อน dispatch เขียน subtask list ออกมาก่อน เช็คว่า
ไม่มีสองงานเขียนไฟล์เดียวกัน แล้วค่อย fork

Tournament — เมื่อคุณภาพสำคัญกว่าความเร็ว รัน 3 agent บน prompt เดียวกัน
แล้วเลือกที่ดีที่สุด ไม่ใช่ pattern สำหรับความเร็ว แต่สำหรับความหลากหลายของ
solution เก็บไว้ใช้กับงานที่ revert ยาก เช่น public API หรือโค้ดที่กระทบความปลอดภัย
แต่ tournament ไม่ใช่เครื่องมือ debug — สาม agent ที่เห็น error message เดียวกัน
มักหลงทางเหมือนกันหมด

Swarm/Workflow subagents — เมื่อนับ task ได้เกิน 100 แล้วเท่านั้น ห้าม spawn
swarm ก่อนที่จะ enumerate task list ได้ นี่คือนิยามที่หนักที่สุด — swarm ที่
dispatch ก่อนมี manifest จะ duplicate งาน, miss งาน, หรือชน shared state กัน
สำหรับงานเบา 100+ ขึ้นที่ I/O isolate ได้ ใช้ workflow subagents ผ่าน queue
สำหรับงานหนักที่ชน rate limit ใช้ five teams rotating ข้าม API key แต่ทั้งสองแบบ
ต้องมี manifest ก่อนเสมอ ไม่มี manifest ไม่มี swarm

โปรเจกต์จริงมักไม่ fit pattern เดียว — discovery ด้วย solo หนึ่งตัว แล้วแบ่ง manifest เป็นกลุ่มตาม pattern ที่เหมาะ กลุ่มงานเล็กจำนวนมากไปที่ workflow subagents กลุ่มงาน critical ไปที่ tournament กลุ่ม architecture ไปที่ trio จบด้วย integration solo หนึ่งตัวรวมทุกอย่างเข้าด้วยกัน — แต่ละชั้นใช้ pattern ชั้นต่ำที่ fit กับ task class ของมันเท่านั้น ไม่มีชั้นไหนได้ overhead เกินจำเป็น

10.2 Command Glossary ที่ใช้บ่อยที่สุด

หาคำสั่งนี้ครอบคลุมวงจรชีวิตของทีมทั้งหมด ตั้งแต่ก่อน spawn จนถึง teardown

```
# 1. Fleet status – เช็คก่อนทุกอย่าง
maw ls -v

# columns: ENGINE, STATUS, TRUST, CODEX_HOME, LAST_SEEN
# ไม่ใช่ OK/TRUSTED – ห้าม spawn

# 2. Preflight – เงื่อนไขเดียวที่ปลอดภัยให้ launch
maw team preflight ψ/teams/<file>.yaml # PATH form – ใช้ตอน script/CI
maw team up trio-alpha                # NAME form – ใช้ตอน
interactive หลัง trust ตั้งแล้ว

# 3. Spawn
maw team up <name>                    # ทั้งทีม ตามลำดับ dependency
maw team up <name> --only <role>      # role เดียว – ใช้ตอน coder ตัว
เดียวตาย ไม่ต้อง respawn ทั้งทีม

# 4. Dispatch
maw hey <session>:<coder> '<task>'

# single-quote task กัน shell expansion
# fire-and-observe ไม่ใช่ fire-and-forget – peek ทันทีหลัง dispatch

# 5. Monitor
maw peek <session>:<coder>
```

```
# read-only tail, exit ด้วย q หรือ Ctrl-C
# ใช้ดี ๆ ใน 5 นาทีแรกหลัง dispatch — ปัญหาที่จับได้นาทีที่สองไม่มีต้นทุน
# ปัญหาที่จับได้ชั่วโมงที่สองกินทั้ง context window ของ coder

# 6. Teardown
maw tmux kill "<session>:<window>"
# ควอดนั้นกัน shell word-splitting บน colon
# kill ก่อน respawn เสมอ — zombie pane ชี้อชนกับ spawn ใหม่ ทำให้ attach เข้า session เก่าโดยไม่รู้ตัว
```

`maw hey` มีจุดที่คนใหม่มักพลาด — ส่งไปแล้วคิดว่าจบ แต่ dispatch เจียบ ๆ ล้มเหลวได้ ถ้า tmux pane scroll เลย injection point หรือ coder ค้างอยู่ที่ interactive prompt วิธีเดียวที่รู้คือ peek ทันที ไม่ใช่รอ

ส่วน `maw hey` ข้าม oracle — ข้อมูลจากภาค 2 — ใช้ address ต่างออกไป แต่หลักการเดียวกัน คือ fire แล้ว observe ไม่ใช่ fire แล้วลืม peer ที่กำลัง busy dispatch งานตัวเองอยู่ก็ยังคงตอบได้ระหว่าง step ของมัน — ไม่ต้องรอ idle ไม่ต้อง share context ล่วงหน้า

10.3 Checklist ก่อน spawn จริง

Checklist นี้รวมจากทั้งสองภาค — foundations (หกขึ้นก่อน spawn) บวก lesson จาก cold consult ในภาค 2 (verify ว่าข้อมูลที่อ้างอิงยังทันสมัย และทุก dependency reachable จริง)

ก่อน spawn ครั้งแรกของ session

1. `maw ls -v` — ทุก engine ต้อง OK/TRUSTED ก่อน
2. Charter YAML ต้องประกาศทุก role พร้อม map ไปยัง engine ที่มีจริง — เช็คด้วย `yq e '.' ψ/teams/<file>.yaml` ก่อน เพราะ YAML พังหน้าตาเหมือน engine error
3. Engine ที่อ้างใน charter ต้อง register ใน maw config แล้ว — `maw engine add` ถ้ายังไม่มี

4. ทุก CODEX_HOME ต้องมี `config.toml` grant trust — ไม่มี trust แปลว่า engine spawn แล้ว halt รอ confirm ที่ไม่มีวันมา
5. Branch แยกสำหรับ session นี้ต้องมีอยู่ — coder ที่ spawn บน main แล้ว commit ตรง คือต้นตอ merge disaster ที่หนักที่สุด
6. `git status` สะอาด, `git stash list` ว่างหรือรู้ที่มา — working tree สกปรกตอน spawn ทำให้ coder ทุกตัวอ่าน status แล้วสับสนว่ามันคือ conflict จริงหรือเปล่า
7. `maw team preflight` ต้อง GREEN — ถ้า non-zero ห้าม spawn แก่ก่อน รันใหม่ จน GREEN ค่อย spawn

เพิ่มจาก cold-start / peer consult (ภาค 2)

8. ถ้ามี peer oracle ที่เพิ่งทำ stack เดียวกันมาสด ๆ ถ้ามก่อนเขียน charter จากความจำหรือ doc เก่า — doc ที่เขียนไว้วันก่อนกับ contract ปัจจุบันของ tool ต่างกันได้เสมอ (เคสจริง — guidebook บอกว่ามี `defaults.worktree` block แต่ contract ปัจจุบันใช้ setup script แยก worktree-local CODEX_HOME แทน)
9. ก่อนถือว่า doc หรือ setup step “reproducible” ให้ list ทุก script/config/binary ที่มันอ้างอิง แล้วเช็ค reachable จากนอกเครื่อง/session ของคนเขียนจริงหรือไม่ — สคริปต์ที่อยู่แค่บนเครื่อง local ไม่เคยเข้า git repo คือช่องโหว่ที่มองไม่เห็นจนมีคนถามว่า “ไฟล์นี้อยู่ไหน”
10. ถ้า tool คื่น error ที่ path ไม่ตรงกับ input (เช่น prefix หายไปเจียบ ๆ) ให้ปฏิบัติเหมือนเป็น bug จริงที่ควร reproduce และ report ไม่ใช่รีบสรุปว่าเป็นความผิด cwd ของตัวเอง — reproduce ข้างบน charter ที่สองอิสระจากกัน แล้ว report มักเร็วกว่านั่ง workaround คนเดียวต่อไป

Smoke test — ก่อน scale ไปทีมใหญ่

ก่อน spawn full fleet เสมอ ต้องมี smoke test อย่างน้อยหนึ่ง agent หนึ่ง task ผ่าน
ก่อน ทีม 20 ตัวที่ spawn พร้อมกันโดยไม่เช็ค prompt ก่อน จะได้ output ผิดแบบเดียว
กันทั้ง 20 ตัว — ตรวจแค่ตัวเดียวถูกกว่าตรวจ 20 ตัวทีหลังเสมอ

10.4 บทส่งท้าย — ทีมถัดไปควรเริ่มจากบทไหน

เล่มนี้แบ่งเป็นสองภาคจริง — foundations ที่ให้ทริตตัดสินใจ pattern และ case study ที่
เล่า cold consult จริงหนึ่งเซสชัน ทีมที่มาอ่านเล่มนี้ครั้งแรกไม่จำเป็นต้องอ่านเรียงจากบท
หนึ่ง คำถามที่ควรถามตัวเองก่อนคือ “ตอนนี้ฉันอยู่จุดไหนของ workflow”

ถ้ายังไม่เคย spawn ทีมเลย ยังไม่มี charter YAML ในมือ — เริ่มที่ 10.3 checklist ก่อน
แล้วย้อนกลับไปดูบทที่อธิบาย charter format ในภาคแรก จะประหยัดเวลากว่าไล่อ่านทุก
บท

ถ้ามีทีมแล้วแต่ไม่แน่ใจว่าควร spawn กี่ตัวสำหรับงานตรงหน้า — กลับมาที่ 10.1 เติมน้ำ

ให้จบก่อน แล้วค่อยเปิด command glossary ใน 10.2 ประกอบ

ถ้ากำลังจะเริ่ม session จาก zero context จริง ๆ — ไม่มี handoff ไม่มีความจำ

session ก่อนหน้า — นี่คือจุดที่ภาค 2 มีค่าที่สุด บทเรียนสำคัญคือ อย่าเชื่อ doc เก่าเกิน

กว่าที่ peer สดจะบอกได้ ถ้ามี oracle อื่นในเครือข่ายที่แตะ stack เดียวกันมาไม่นาน ถาม

ก่อนเขียน อย่าเขียนจากความจำ

สิ่งที่ทีมถัดไปควรทำต่อ ไม่ใช่แค่ทริตตัดสินใจให้ขึ้นใจ แต่คือการวัด — ทุกครั้งที่เลือก

pattern ให้บันทึกว่าทำไมเลือก แล้วหลัง teardown เทียบว่า choice นั้นถูกจริงไหม ตัว

เลขจาก session หนึ่งไม่ใช่กฎสากล มันเป็นแค่ calibration point เดียว การ scale จาก

solo ไป trio ไป swarm ไม่ใช่ความสำเร็จของ crew master — ทีมที่เลือก solo ถูกต้อง

สำหรับงานเล็ก และไม่เคยมองแต่ swarm เลยทั้งโปรเจกต์ คือทีมที่ทำถูกพอ ๆ กับทีมที่

บริหาร swarm ร้อยตัวได้สำเร็จ

การตัดสินใจไม่ได้อยู่ที่ scale สูงสุดที่ทำได้ แต่อยู่ที่ scale ที่พอดีกับงานตรงหน้า — เลือก

ทีมที่เล็กที่สุดที่พิสูจน์ pattern ได้ แล้วค่อยขยายเมื่อข้อมูลบอกว่าจำเป็นจริง ไม่ใช่เพราะ

มันดูน่าประทับใจกว่า